Yaksha-Prashna: Understanding eBPF Bytecode Network Function Behavior

Animesh Singh
Indian Institute of Technology Hyderabad

K Shiv Kumar
Indian Institute of Technology Hyderabad

S. VenkataKeerthy
Indian Institute of Technology Hyderabad

Pragna Mamidipaka
Carnegie Mellon University

R V B R N Aaseesh
Indian Institute of Technology Hyderabad

Sayandeep Sen
IBM Research India

Palanivel Kodeswaran
Walmart Global Technology Labs

Theophilus A. Benson
Carnegie Mellon University

Ramakrishna Upadrasta
Indian Institute of Technology Hyderabad

Praveen Tammana
Indian Institute of Technology Hyderabad

Abstract

Many cloud infrastructure organizations increasingly rely on third-party eBPF-based network functions for use cases like security, observability, and load balancing, so that not everyone requires a team of highly skilled eBPF experts. However, the network functions from third parties (e.g., F5, Palo Alto) are available in bytecode format to cloud operators, giving little or no understanding of their functional correctness and interaction with other network functions in a chain. Also, eBPF developers want to provide proof of functional correctness for their developed network functions without disclosing the source code to the operators.

We design Yaksha-Prashna, a system that allows operators/developers to assert and query bytecode's conformance to its specification and dependencies on other bytecodes. Our work builds domain-specific models that enable us to employ scalable program analysis to extract and model eBPF programs. Using Yaksha-Prashna language, we express 24 properties on standard and non-standard eBPF-based network functions with 200-1000x speedup over the state-of-the-art work.

1 Introduction

Today, the cloud infrastructure of many large and prominent organizations [17, 34, 41, 65] rely on eBPF-based network functions (NFs) for critical management functionality (e.g., security [22], observability [18], and load balancer [64]). These eBPF network functions have been the cause of several high profile outages, e.g., Datadog's 5 Million Dollar outage [37, 69] or outages discussed in Meta's NetEdit [16]. While eBPF has been adopted because of its programmability, high performance, and safety guarantees, eBPF development requires a team of highly-skilled experts [13], forcing many organizations to rely on eBPF programs from third parties (e.g., F5's DoS protect [7], Palo Alto's firewall [10], Datadog [29], Cilium [18], and Meta's load balancer (katran) [64]). Due to recent large-scale outages caused by such third-party NFs, there is a growing concern about the functional correctness of these eBPF programs [47, 55, 61].

Unfortunately, understanding the functional correctness and their interaction with other NFs is non-trivial because it requires understanding program behavior which varies from kernel to kernel [16], requires analyzing program code which includes many BPF-specific constructs and lastly documentation of the eBPF language and programs are poor. Even advanced developers, struggle because understanding NF behavior requires both interactions between the NF and other NFs deployed along side but also with the kernel. More importantly, to check an NF's conformance with its specification, it is crucial to know the network context, such as read/write/copy operations on packet content, protocols processed, and eBPF maps invoked. The bytecode's network context also helps to understand its interaction with other NFs (*e.g.*, Write before Read on a packet header). Thus, while understanding eBPF-based functions gives operators confidence to debug or avoid outages before deployment in production networks [24, 36], they lack such knowledge and may have to wait for a longer duration [31] for support from third-party eBPF program developers.

In this paper, we take a classic diagnosis approach to the problem: we aim to hide the nuances associated with kernel versions, eBPF specification, detailed knowledge of program design and interactions by introducing a query language that *enables network operators and developers to assert and query eBPF-based NFs from their bytecode and understand their behavior.* Our query language enables a broad set of

analysis through simple language constructs (e.g., pre/post-conditions [42]) and language extensions (e.g., writing assertions/contracts [80]). To support this query language we introduce a program analysis-based (e.g., program verification [33, 44] and abstract interpretation [27, 28]) framework that builds models that infer a program's behavior. Our work differs from prior work on diagnosis because of our need to address eBPF specific constructs, e.g., helpers and maps, which make eBPF bytecode sufficiently different from traditional programs. To this end, we need to address two novel challenges: an eBPF-centric static analysis model, and an eBPF-centric query system to support user interaction.

Behavioral model for analysis. Existing works on NF verification rely on formal models of the source code for verifying the properties of the NFs implemented in high-level languages (like C and P4). For instance, tools like eBPF-SE [47] model eBPF programs and leverage the generic symbolic execution engines like Klee [23] to analyze these models. However, the applicability of such works to eBPF bytecode is limited due to a lack of behavioral models on eBPF bytecode. On the other hand, there are eBPF verifiers [40, 82] that operate on bytecode, thus alleviating the need for source code, but they mainly model the semantic properties to validate program termination, memory safety, and resource boundedness, using static analysis [82], abstract interpretation [40] and symbolic execution [70] techniques. These verifiers are not designed to extract network context from bytecode.

Extracting network context from a bytecode (*e.g.*, protocol processed) introduces two key challenges over the standard source-code analysis: First, eBPF bytecode instructions are at a low level, similar to assembly. Each high-level statement can translate into multiple bytecode instructions, making it challenging to interpret and understand the relation to other instructions in the execution flow. Second, due to the limited number of registers and stack memory, large and complex eBPF programs reuse registers and memory locations, leading to frequent redefinitions that further obscure the analysis.

Addressing these challenges, we design **Yaksha-Prashna Analyzer**, which analyzes bytecode based on the classic dataflow analysis [43], guided by control flow analysis (§4.1). The analyzer captures granular information at each program point by carefully tracking data flow across registers and memory locations (§4.2). We define dataflow rules to extract the network context of the eBPF bytecode effectively (§4.3). Interestingly, the characteristic features of eBPF, like limited state space, absence of loops, and a fixed number of registers, etc., make our analyzer both efficient and scalable.

Behavioral domain-specific language (DSL). Another requirement is the user interaction by expressing behavioral queries on the eBPF bytecode. Existing tools for NF verification take the standard program verification approach and primarily focus on extending the (source) language with assertions [12, 38, 61, 67]. These extended assertion languages have limited scope; Assertions are good at validating if certain properties hold (e.g., "*Does the NF write to a map?*") but cannot retrieve the network context (e.g., "*Which packet header field is written to the map by NF?*"). Furthermore, each assertion typically requires its separate analysis pass; therefore, for each new assertion, the program is analyzed from scratch to validate those additional properties, thereby incurring significant computational overheads (both in analysis time and memory consumption).

Consequently, there is a necessity for a custom query language that decouples the retrieval of network context from querying the extracted network context. Designing this query language poses two main challenges: First, the language should effectively be able to map the high-level intention of NF operators/developers to the low-level constructs in the bytecode while effectively abstracting out the low-level details in the bytecode. Second, the language should be able to express complex queries that check conformance and also bytecode's interaction with other NFs.

Addressing these challenges, we design **Yaksha-Prashna Language**, a simple but expressive DSL, to abstract the complexities of the analysis. Yaksha-Prashna allows network operators/developers to express high-level queries about NF behavior without lower-level bytecode details like the basic block IDs, register mappings, etc. Yaksha-Prashna supports users to *express* two types of queries, assertion and retrieval, on individual bytecode and across multiple bytecodes for cross-function interactions. It is *composable*, allowing users to combine simple queries to formulate complex real-world queries, and *adoptable*, leveraging Prolog for easy logic-based querying.

To ensure *scalability* of our system, we decouple the bytecode analysis from the query phase, allowing the bytecode to be analyzed once, with the extracted network context reused to answer any subsequent queries. This approach eliminates repeated analysis passes, reducing overhead and supporting efficient, large-scale deployment.

The key contributions of our paper are as follows:

- We propose Yaksha-Prashna Language, a domain-specific query language for NF operators/NF developers to assert and retrieve network context of eBPF bytecode, abstracting the complex dependencies among instructions in low-level bytecode (§5).
- We construct a formal dataflow analysis model, guided by control flow analysis, for eBPF bytecodes and design Yaksha-Prashna Analyzer to extract network context (§4).
- We present Yaksha-Prashna (§3), a comprehensive system that integrates the analyzer (§4) and the query language (§5). The system is scalable as it decouples the bytecode analysis from the query execution phase.

- We prototype Yaksha-Prashna and tested by executing queries on 16 XDP programs from projects like Katran [64] and Suricata [35].
  The analyzer extracts network context of bytecode with 4K paths in 300 *ms* while consuming less than 50 MB of memory. Yaksha-Prashna's query system executes 15 queries on 16 XDP programs in less than 50 μs while consuming less than 15 MB of memory.
- We compared Yaksha-Prashna with state-of-the-art tools, Klint [70] and DRACO [61], based on expressivity of the language and verification time. Yaksha-Prashna can express the properties used for verification while keeping the verification time 200-1000× less.

2 Background and Motivation

2.1 Third-party eBPF network functions

Many organizations rely on third-party eBPF programs. While many are open-source (*e.g.*, Data-dog's NPM, Cilium's CNI, Meta's Katran), a growing number of observability and security programs are proprietary, and their bytecodes are distributed to operators either directly [9, 10] or through marketplaces like L3AF [55]. Some example closed and proprietary eBPF-based NFs are NGINX App protect DoS from F5 Networks [7] and CN-series firewall by Palo Alto [10]. We see this is in line with the marketplace paradigm for virtual network functions (VNF) like equinox network edge [6] which offers VNFs from various vendors like Cisco, Juniper, Palo Alto, and Fortinet.

Why understanding bytecode is important? Typically, an eBPF-based NF is developed in a high-level language [32, 46, 51, 63, 74, 79] and is compiled into bytecode using Clang/LLVM toolchain [56]. The bytecode is loaded into the host kernel with the help of system calls [76]. During this process, the eBPF verifier performs static checks on the bytecode to ensure that the bytecode runs safely inside the kernel. However, the *verifier does not model or reason the functional correctness of the bytecode.* More specifically, the third-party eBPF-based NF may not be buggy, but it may not fully cover the target behavior [61], that is, functional correctness. This can happen due to various reasons, such as missing statements (e.g., a missing TTL decrement in a router [81]), unexpected packet forwarding behavior (e.g., router forwarding packets with TTL = 0), or deviations from standard NF behavior (e.g., a firewall unintentionally allowing unknown traffic), etc. Also, the NF's interactions with other NFs in the system can cause unexpected behavior. If the programs are from eBPF marketplaces [55], they may have unknown or malicious code [61]. This motivates the need to understand the bytecode's packet-processing behavior and its interactions with other NFs before deploying it in the network.

Intented users of Yaksha-Prashna. Network operators and network developers are the two main users of the proposed tool. **(1) Network operators:** The documentation and

```
Query                  ::=  Q-Predicate_list '.'
Q – Predicate_list     ::=  Q-Predicate [operator Q-Predicate]*
Q – Predicate          ::=  '!''('Q-Predicate')'
                        |   field_pred'(' nf_id ',' fld_arg')'
                        |   pkt_act_pred'(' nf_id ',' hook_arg ',' list_of_pairs')'
                        |   map_operation_pred'('nf_id','map_id','fld_arg')'
                        |   correlated_maps_pred'('nf_id','list_of_maps')'
                        |   order_pred'(' nf_id ',' var')'
                        |   protocol_pred'(' nf_id ',''fld_arg ','value')'
                        |   helper_pred'(' nf_id ',' helpers')'
operator               ::=  ',' | ';'
list_of_pairs          ::=  '['pair [operator pair]*']'
list_of_maps           ::=  '['('map_id ',' map_id ')']'
pair                   ::=  '('fld_arg ',' value ')'
field_pred             ::=  updatesField | readsField
pkt_act_pred           ::=  drops | passes | aborts | redirects | tx | all
map_operation_pred     ::=  mapLookup | mapWrite
correlated_maps_pred   ::=  correlatedMaps
order_pred             ::=  successorNF | predecessorNF
protocol_pred          ::=  accessesProtocol
helper_pred            ::=  callsHelper
fld_arg                ::=  header_field | buffer_field | var | '*'
hook_arg               ::=  xdp | tc | var | . . .                    // Hook points
header_field           ::=  eth.type | eth.dst | eth.src | . . .
                        |   ipv4.src | ipv4.dst | . . .
                        |   . . .                                     // Std header fields
buffer_field           ::=  xdp_md.data | xdp_md.data_end | . . .
                        |   sk_buff.mark | . . .
                        |   . . .                                     // Kernel Buffer fields
helpers                ::=  bpf_map_lookup_elem
                        |   bpf_map_update_elem
                        |   bpf_probe_write_user| var | . . .
                        |   . . .                                     // eBPF helper functions
nf_id                  ::=  string | var | '*'
map_id                 ::=  string | var | '*'
value                  ::=  const | var | '*'
var                    ::=  [A-Z_][A-Za-z0-9_]*                     // Variable
const                  ::=  !const | [0-9]+ | string | ip-address
```

Figure 1. Grammar for Yaksha-Prashna Language.

specification for closed bytecodes give little visibility into the inner workings [70]. An NF's specification informs what the NF can do, but does not specify how the NF should be implemented. If the NF conforms to the respective specification, then the operator is confident to deploy it, even though access to the source code is restricted. Moreover, operators may want to verify that integrating new NF will not cause disruptions or unintended interactions within the existing NF chains. Even when source code is provided, as with Cilium, understanding such behaviors requires tedious and significant efforts. Yaksha-Prashna enables operators to validate the functional correctness of NFs to be deployed safely in the existing NF chain. **(2) Network developers:** The developers want to guarantee the functional correctness of their developed NFs, without disclosing the source code, to the operators. Moreover, bytecode conformance to specification enables developers to use any language and toolchain, especially those in the experimental phase, so that the compiled binary satisfies the target behavior [70]. Yaksha-Prashna helps the developers in achieving these goals.

2.2 Motivating use cases

A usable verification system should meet three requirements [81] for users so that they can: (1) express the correct NF behavior (specification) with ease, (2) check the conformance of the NF/NF chain within a reasonable time and low memory overhead, and (3) reason about the underlying causes of specification violations. Yaksha-Prashna language (Figure 1)

is designed to meet all three, so that NF operators/NF developers can validate bytecode using assertions and understand the NF behavior using retrieval queries. Details of the language are in Section §5.

We categorize the use cases of Yaksha-Prashna into three.

Category 1: Individual NF conformance to specification. Consider that a user (NF operator/NF developer) wants to validate whether an NF implementation (bytecode) conforms to its specification. For example, a stateful firewall specification says (1) it should not modify packet contents; and (2) it should allow only TCP and UDP traffic, and drop other traffic (*e.g.*, ICMP [61]). Listing 1 shows an example stateful firewall code snippet that does not conform to the specification; it updates the source port (line 10) of TCP traffic and processes ICMP traffic (line 6). They must know that the bytecode generated from this eBPF program does not conform to the specification. To do so, they can write the following assertions to check whether the NF bytecode conforms to the specification.

The assertion below asserts that the firewall bytecode (Listing 1) does not modify (update) packet content.

A1: `!updatesField(xdp_fw, *).`

It gets the list of packet fields updated by the bytecode and returns false if the list is not empty. The argument $*$ indicates that this is an assertion. The assertion fails on the firewall because it modifies the TCP source port.

The assertion below checks that the firewall bytecode at the XDP hook point does not allow the ICMP traffic.

A2: `passes(xdp_fw, xdp, [(var, var)]), !accessesProtocol(xdp_fw, “ipv4.proto”, “icmp”).`

passes construct with (*var*, *var*) as an argument returns the network context of all program paths that passes the packet. Next, it checks that these paths should not access the ICMP protocol headers, using !*accessesProtocol* construct. The assertion fails on the bytecode generated from Listing 1 because it processes ICMP traffic (line 6), and passes the packet.

Category 2: Unintended NF interactions. Multiple eBPF programs are attached to the same hook point (*e.g.*, XDP, TC). This enables the development of modular programs to build complex systems. However, an update to a state by one program can lead to unexpected behavior if another program in the chain relies on that state. More specifically, some dependencies that an operator may want to know are Read after Write (RAW), Write after Read (WAR), and Write after Write (WAW) [61]. To illustrate this further, consider an issue observed in practice that resembles RAW dependency. A bad interaction between Cilium and AWS elastic network interface [24, 36] resulted in an unintended drop in traffic for some customers. This interaction is illustrated using a code snippet in Listing 2. Here, Cilium's *identity_manager* (NF1), stores security ID *id_prefix* in *sk_buff->mark* field to represent the traffic from a specific pod in a specific cluster.

```
1  int xdp_fw(struct xdp_md *ctx) {
2  ...
3  switch(ip->protocol){
4    case IPPROTO_TCP :  goto l4;
5    case IPPROTO_UDP :  goto l4;
6    case IPPROTO_ICMP : goto l4; /* Parses ICMP */
7    default :           goto EOP;
8  }
9  ...
10 tcp->sourceport = 8080; /* updates TCP source port */
11 ...
12 if(ingress_ifindex == EXTERNAL){
13   flow_leaf = bpf_map_lookup_elem(&flow_ctx_table, &
         flow_key);
14   if(flow_leaf)
15     return bpf_redirect_map(&tx_port, flow_leaf->
           out_port, 0);
16 ... }
17 else{
18 flow_leaf = bpf_map_lookup_elem(&flow_ctx_table, &
       flow_key);
19     if (!flow_leaf){ ...
20       bpf_map_update_elem(&flow_ctx_table, &flow_key,
             &new_flow, 0);}
21 ...}
22 EOP : return XDP_DROP;
23 }
24 \
```

Listing 1. Firewall code snippet

```
1     ********** Cilium NF (NF1) **********
2  int identity_manager(struct __sk_buff *skb) {
3    __u32 *secid = bpf_map_lookup_elem(&sec_id_map, &
         key);
4  ...
5    /* Extract first 8 bits (most significant byte) */
6    __u8 id_prefix = (*secid >> 24) & 0xFF;
7    if(skb->mark){
8      /* Updates skb->mark field */
9      skb->mark = (skb->mark & 0xFFFFFF00) | id_prefix;
10   }
11 ... }
12
13    ********** AWS NF (NF2) **********
14 int pbr_routing(struct __sk_buff *skb) {
15   ...
16   *masks = bpf_map_lookup_elem(&mask_map, &key);
17   ...
18   __u32 new_mark = (nfmark & ~nfmask) ^ (CTMARK &
         ctmask);
19   skb->mark = new_mark; /*Updates skb->mark field*/
20   ... }
21
22    ********** Cilium NF (NF3) **********
23 int enforce_policy(struct __sk_buff *skb) {
24   ...
25   /* Retrieves Cluster_id from skb->mark field */
26     __u32 cluster_id = skb->mark & 0xFF;
27     __u32 *val = bpf_map_lookup_elem(&cluster_map, &
           cluster_id);
28   /* Cluster ID not found, drop the packet */
29     if (!val) { return TC_ACT_SHOT;}
30     /* Cluster ID is known, allow packet */
31     return TC_ACT_OK;
32 }
```

Listing 2. Unintended write by AWS's NF2 before read by Cilium's NF3 lead to traffic drop.

Down the chain, another NF from cilium, *enforce_policy* (NF3), does a lookup on *cluster_id* in *sk_buff->mark* (line 26-32) and allows traffic. However, *pbr_routing* (NF2) at the AWS elastic network interface, introduced newly into the packet path, updates *sk_buff->mark* (line 19) to capture the connection type (default). As a consequence, *cluster_id* is

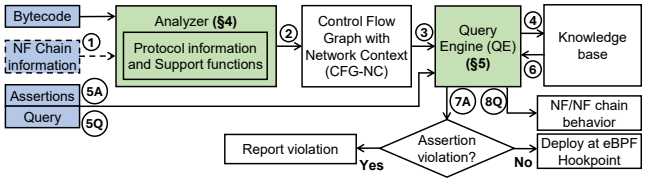


Figure 2. Yaksha-Prashna system workflow

corrupted, and traffic for some customers is dropped. To avoid such bad interactions before they happen, it is crucial to know dependencies before a bytecode is introduced into a chain.

The following assertion checks for Read After Write (RAW) dependency between NF2 and any successor NFs (NF3) in the chain, as shown in Listing 2. This assertion can be executed on the new chain before deploying NF2's bytecode.

A3: `!updatesField(NF2, *), successorNF(NF2, SNf), readsField(SNf, *).`

updatesField with $*$ argument retrieves a list of fields updated by NF2's bytecode (write list), including socket buffer and packet headers. *successorNF* gets all the successor NFs of NF2, and *readsField* construct retrieves the list of fields read by them (read list). The assertion fails if the write list overlaps with the read list of any of NF2's successors in `SNf`. Yaksha-Prashna language also supports assertions to check for other dependencies (WAR, WAW).

Category 3: Understanding the NF/NF chain behavior. Once it has been established that the NF bytecode does not conform to its specification, or that a dependency exists in an NF chain, identifying the cause enables precise and reliable bug reporting. For example, one can accurately document and report that the specification violation in Listing 1 was due to the modification of the TCP source port, and *sk_buff->mark* field is responsible for the dependency in Listing 2.

In the first case, the retrieval query

RQ1: `updatesField(xdp_fw, Fld).`

retrieves a list of packet fields (*e.g.*, *tcp.sport*) updated by the firewall, stores them in *Fld*, and returns as the query result.

In the second case, the following retrieval query can be used to debug the assertion A2's failure.

RQ2: `updatesField(NF2, Fld), successorNF(NF2, SNf), readsField(SNf, Fld).`

It retrieves the list of socket buffer and packet header fields updated by NF2 and read by any of NF2's successors, stores them in *Fld*, and returns them as the query result (*i.e.*, *sk_buff->mark* in Listing 2). To summarize, retrieval queries provide deeper insight into packet processing behavior, such as protocols being accessed, header/buffer fields being read/updated, shared map between NFs, helper functions invoked, etc.

3 Yaksha-Prashna overview

Yaksha-Prashna workflow is presented in Figure 2. It consists of two main components: the **Analyzer** (§4) and the **Query Engine (QE)** (§5). **(1)** For a given bytecode (optionally accompanied by NF chain information, if the user wants to check for unintended interactions before deploying a new NF), the analyzer builds a formal dataflow analysis model guided by control flow analysis. This model is represented as a control flow graph (CFG) and annotated with the network context (NC) extracted from the bytecode, resulting in **(2)** the CFG-NC. We define the bytecode's network context as the set of read/write/copy operations on packet content (headers and metadata), the network protocols processed, the eBPF maps accessed, and the helper functions invoked by the bytecode. **(3)** The Query Engine (QE) translates the network context encoded in the CFG-NC into **(4)** the Knowledge Base (Prolog facts), while preserving the structural integrity of the CFG-NC. **(5A)** Assertions written in the language (Figure 1) are evaluated by the QE by **(6)** mapping their predicates to the corresponding facts stored in the Knowledge Base. **(7A)** If an assertion violation is detected, it is reported; otherwise, the NF can be deployed at the eBPF hook point. In the case of a violation, **(5Q)** retrieval queries are issued to the QE, and **(8Q)** the behavior of the NF or NF chain is retrieved.

4 Yaksha-Prashna Analyzer

Dataflow analysis for network context extraction. To extract network context, we depend on the widely used classic data flow analysis and propagation [43, 48, 49, 52] due to its scalability and comprehensive coverage of programs. Alternatives like SAT/SMT solvers [30] and abstract interpretation [27], while being powerful for exhaustive state spaces and offer various benefits verifying program properties, come with significant limitations: primarily, they are computationally expensive [26, 28, 39], and may not be well-suited for retrieving precise, detailed information about program behaviors, especially about bytecodes.

In contrast, data flow analysis is well-suited for eBPF bytecode, which is simpler than traditional programs due to its lack of loops and fixed register count. This simplicity reduces the state space, making the analysis more scalable and manageable, where even complex NFs are handled with minimal overhead. Data flow analysis allows for granular insights into the network context at each program point, offering a more efficient and detailed approach.

Challenges. As discussed in §1, two key challenges arise in extracting network context information at the eBPF bytecode level. This involves (1) interpreting instructions' semantics in relation to another, and (2) tracking dataflow information across register and memory locations involving redefinitions.

Overview. To address these challenges, we propose a formal flow analysis framework that integrates both register/memory state tracking and context state extraction. Particularly, we define formal context state (C) rules that map low-level bytecode instructions to high-level program semantics. These rules capture important operations, such as reading packet

Table 1. Transfer function rules.

Rule	Instruction	Condition	State transformation	
			Reg. and Mem. states	Context state
R1: Arithmetic operation	$dst \leftarrow dst \oplus imm \mid src$	–	$\mathcal{R}[dst].val \leftarrow \mathcal{R}[dst].val \oplus imm \mid \mathcal{R}[src].val$	–
R2: Assignment operation	$dst \leftarrow src \mid imm$	–	$\mathcal{R}[dst].tag \leftarrow \mathcal{R}[src].tag$ | **Const**, $\mathcal{R}[dst].val \leftarrow \mathcal{R}[src].val \mid imm$	–
R3: Writing to packet buffer	$*(dst + off) \leftarrow imm \mid src$	$\mathcal{R}[dst].tag ==$ **pkt_buff**	–	$C[write\_buff].val \mathrel{+}= [\text{getBuffFldName}_{spec}(buff\_type, off), imm \mid \mathcal{R}[src].val]$
R4: Writing to packet data	$*(dst + off) \leftarrow imm \mid src$	$\mathcal{R}[dst].tag ==$ **pkt_data_start**	–	$C[write\_hdr].val \mathrel{+}= [\text{getHdrFldName}_{spec}(C[curr\_proto].val, off), imm \mid \mathcal{R}[src].val]$
R5: Writing to program stack	$*(dst + off) \leftarrow imm \mid src$	$\mathcal{R}[dst].tag ==$ **stack_frame**	$\mathcal{M}[off].tag \leftarrow$ **Const** $\mid \mathcal{R}[src].tag$, $\mathcal{M}[off].val \leftarrow imm \mid \mathcal{R}[src].val$	–
R6: Accessing map	$dst \leftarrow map\_by\_fd(imm64)$	–	$\mathcal{R}[dst].tag \leftarrow \textbf{Ref}_{\text{map}}$ $\mathcal{R}[dst].val \leftarrow imm64$	$C[map\_read].val \mathrel{+}= \text{getMapName}_{elf}()$
R7: Writing into map	$CALL\ 2$	$\mathcal{R}[1].tag == \textbf{Ref}_{\text{map}}$	–	$C[map\_write].val \mathrel{+}= [\text{getMapName}_{elf}(), \text{getHdrFldName}_{spec}(C[curr\_proto].val, \mathcal{R}[3].val)]$
R8: Reading packet buffer	$dst \leftarrow *(src + off)$	$\mathcal{R}[src].tag ==$ **pkt_buff**, $off == \text{getBuffOffset}_{spec}(buff\_type,$ **data** | **data_end** | <**other_field**>)	$\mathcal{R}[dst].tag \leftarrow$ **pkt_data_start** | **pkt_data_end** | **pkt_buff**, $\mathcal{R}[dst].val \leftarrow off$	$C[read\_buff].val \mathrel{+}= \text{getBuffFldName}_{spec}(buff\_type, off)$
R9: Reading packet data	$dst \leftarrow *(src + off)$	$\mathcal{R}[src].tag ==$ **pkt_data_start**	$\mathcal{R}[dst].tag \leftarrow \mathcal{R}[src].tag$, $\mathcal{R}[dst].val \leftarrow (\mathcal{R}[src].val + off)$	$C[read\_hdr].val \mathrel{+}= \text{getHdrFldName}_{spec}(C[curr\_proto].val, off)$
R10: Reading program stack	$dst \leftarrow *(src + off)$	$\mathcal{R}[src].tag ==$ **stack_frame**	$\mathcal{R}[dst].tag \leftarrow \mathcal{M}[off].tag$, $\mathcal{R}[dst].val \leftarrow \mathcal{M}[off].val$	–
R11: Next protocol check	$dst \neq imm \mid dst == imm$	$\mathcal{R}[dst].tag ==$ **pkt_data_start**, $\text{isTailField}_{spec}(C[curr\_proto].val, \mathcal{R}[dst].val)$	–	$C[next\_proto].val \leftarrow \text{getProtoName}_{spec}(imm)$
R12: Memory bound check	$dst < src$	$\mathcal{R}[dst].tag ==$ **pkt_data_start**, $\mathcal{R}[src].tag ==$ **pkt_data_end**	–	$C[curr\_proto].val \leftarrow C[next\_proto].val$
R13: Helper function invocation	$CALL\ imm$	–	–	$C[helper].val \mathrel{+}= \text{getHlprFuncName}_{spec}(imm)$
R14: Return action	$return\ r_0$	–	–	$C[pkt\_action].val \mathrel{+}= [hook, \text{getActionName}_{spec}(\mathcal{R}[0].val), \text{getPathAndNwContext}()]$
R15: Correlated maps	$CALL\ 1 \mid 2 \mid 3 \mid 51$	$\mathcal{R}[2].tag == \textbf{Ref}_{\text{map}}$	–	$C[correlated\_maps].val \mathrel{+}= [\text{getMapName}_{elf}(), \text{getMapName}_{elf()}]$

headers or writing to maps, and protocol-specific behaviors, allowing us to accurately track how context evolves as the instructions are traversed (more details in Table 1 and §4.3). Also, we develop precise register (R) and memory (M) state rules to track the flow of data and interactions throughout the program. These rules handle the complexities of register (re-)definitions and memory updates, ensuring accurate tracking of data flow (more details in Table 1 and §4.2).

Both the context state (C) and the register/memory state (R, M) are governed by a unified flow analysis framework. The register and memory states provide information for updating the context states. Context states are used to derive the bytecode's network context.

4.1 Control Flow Graph with Network Context

Basic block state and network context ($\mathcal{R}$, $\mathcal{M}$, and C). Our dataflow analysis also considers the control flow of the program, by making use of the Control Flow Graph. The absence of complex control flows in the eBPF code simplifies the control flow analysis. We now explain the process of constructing a Control Flow Graph with Network Context (CFG-NC) for a given bytecode. The CFG-NC has *basic blocks* (BB) as nodes, and the edges capture the data propagated between two nodes. We group instructions into sequences that have a single entry point and a single exit point, ending either at a *jump* instruction or just before a *jump target*, and associate each such sequence with a basic block (BB). The instructions in each BB are processed to generate two types of data: BB's program state ($\mathcal{R}$ and $\mathcal{M}$) and BB's network context C. More specifically, the program state captures the state of registers ($\mathcal{R}$) and stack memory ($\mathcal{M}$) after processing a BB. The network context (C) captures the BB's packet-processing behavior, such as read/write/copy operations on packet headers/buffers, maps accessed, and helper functions invoked. More details on how $\mathcal{R}$, $\mathcal{M}$, and C are generated in the following sections (§4.2 and §4.3).

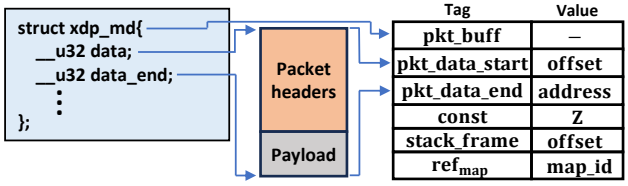


Figure 3. R and M <tag,value> pairs. Z means integer.

State propagation and CFG-NC generation. We design a forward analysis; $\mathcal{R}$, $\mathcal{M}$, and C[1] state of a BB ($OUT[BB]$) forms the IN state ($IN[BB]$) to all its successors in the CFG. A BB's OUT state is computed as a *function* of IN state where the *function* applies a set of state transformation rules (as shown in Table 1) on instructions in the BB. More details on the instruction categories and the rules applied to each category are in Section §4.3. If a BB has more than one predecessor, the OUT state of such predecessor is merged using a *confluence* operator and prepares IN set. That is,

$$IN[BB] = \bigcup_{p \in pred(BB)} OUT[p]$$

The BBs in CFG are processed in depth-first search (DFS) order and the network context (C) extracted from each BB is annotated. Subsequently, the query engine module answers user queries by processing C in the CFG-NC (§5).

4.2 $\mathcal{R}$, $\mathcal{M}$, and C data structures

We present $\mathcal{R}$, $\mathcal{M}$, and C data structures followed by state transformation rules (Table 1) applied on instructions.

R and M data structures. Instructions in eBPF bytecode use up to 11 registers and a program stack of 512 bytes. Complex eBPF programs may run out of registers and often use the program stack whenever register spills are observed [50]. Therefore, analyzing some instructions requires tracking and updating $\mathcal{R}$ and $\mathcal{M}$. $R[n]$ represents register states where $n \in \{0, 1, 2, \ldots, 10\}$, one each for 11 registers (r_0-r_{11}) used in instructions. $M[k]$ represents k^{th} byte of the program stack, where $k \in \{0, 1, 2, \ldots, 511\}$. Each entry r_n and byte k has a (*tag*, *value*) pair. Based on possible values, we came up with six tags as shown in Figure 3: (i) tag **pkt_buff** indicates r_n or byte k holds a reference to a packet struct buffer (e.g., *xdp_md*, *__sk_buff*) with value as none, (ii) **pkt_data_start** contains an offset from the start of packet data, (iii) **pkt_data_end** refers to the end of the packet data, (iv) **const** holds the constant value, (v) $\mathbf{ref}_{\mathbf{map}}$ refers to a eBPF map in the program, and (vi) **stack_frame** refers to a location in the stack.

Network Context (C) data structure. Similar to $\mathcal{R}$ and $\mathcal{M}$, C also has a list of (*tag*, *value*) pairs. There are seven tags: (i) **hdr_field** and **buff_field** capture the list of packet header and packet buffer fields accessed or updated; (ii) **helper_function** captures the list of helper functions invoked; (iii) **map** captures the names of eBPF map accessed; and (iv) [**next_proto**, **bound_check**, **curr_proto**], all three help to extract protocols processed in the bytecode (§4.3).

Initialization: Conventionally, register $r1$ points to the packet buffer at the beginning of the eBPF bytecode – packet buffer structures are defined in the kernel header `bpf.h` (see Figure 3). These structures provide input context to the eBPF programs and hold the pointers to the start and end of the packet [84]. Hence, the state of register $r1$ is initialized to (*pkt_buff*, -), and other states ($\mathcal{R}$, $\mathcal{M}$, and C) are set to NULL.

4.3 Network context extraction

State transformation rules by instruction category. We apply transfer function rules to (i) *propagate* $\mathcal{R}$ and $\mathcal{M}$ states from one instruction to another within and across BBs, and (ii) *record* C in each BB. An instruction in bytecode [1] contains opcode (8 bits), source and destination register (4 bits each), offset (16 bits), and immediate value (32 bits). We divide instructions into 8 categories, as shown in Table 1, and apply associated rules. Each rule has an optional condition, and if the condition is satisfied, R and M states are updated, and the associated network context is recorded in C. Variables are italicized (e.g., $\mathcal{R}[src].tag$, $\mathcal{R}[src].val$, imm, etc.) and the tags are specified in bold (e.g., **pkt_buff**, **pkt_data_start**).

Protocol information and support functions. Bytecode instructions contain immediate values and offsets, each with a specific meaning. Such as field offsets in packet headers or packet buffers (*e.g.*, sk_buff), protocol number (*e.g.*, ethrType, ipv4.protocol), helper function ID, action ID specific to a hook point, and map ID. We capture these IDs in a file and define a set of support functions that perform look-up on the file and return names (*e.g.*, protocol name, helper function name) that an end user can understand. It can be extended to support new protocols, hook points, and helpers.

Running examples. For simple arithmetic (R1) and assignment (R2) operations, we update R and M states, but do not record network context because these instructions would not directly update/read packet headers/buffers. In contrast, we record network context for write operations (R3-R5) and read operations (R8-R10). For instance, in Figure 4, we show bytecode instructions 1, 2, and 15 are read operations and transfer function rules R8, R9, and R9 are applied, respectively[2]. Instruction 11 is of write type, so rule R4 is applied[3].

R6, R7, and R13 are applied for map lookup, map update, and helper functions, respectively. For instance, consider bytecode instructions 3 and 4 in Figure 4. These instructions call the lookup helper function on the *cpus_count* map. On these instructions, R6 and R13 are applied to extract the names of the map and the helper function invoked. Similarly,

[1]We do not propagate the complete network context state, but only a portion that requires previous block's network context

[2]'getBuffOffset' support function get the offset to a packet buffer field as defined in the spec.

[3]'getHdrFldName' and 'getBuffFldName' support functions return the names of the header field and packet buffer, respectively.

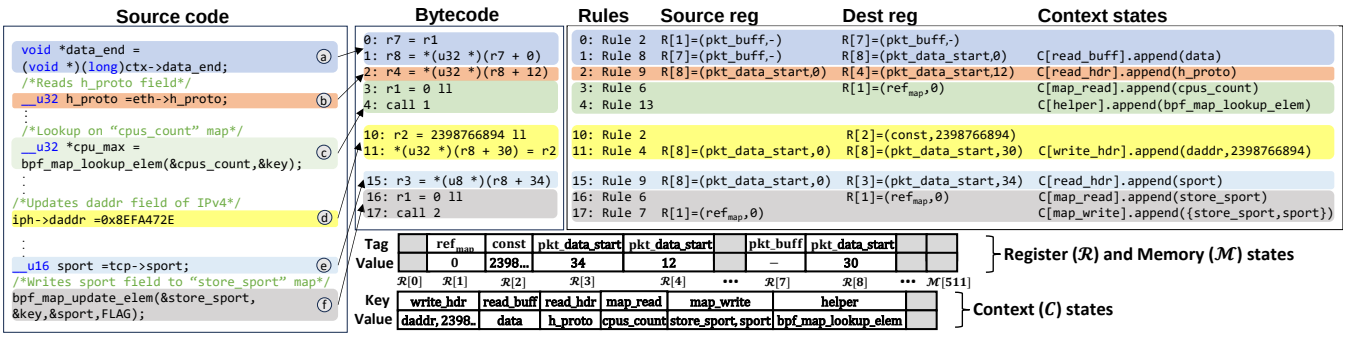


Figure 4. Operation ⓐ reads *data* field from packet buffer. Operation ⓑ reads *Ethertype* (h_proto) field from ethernet header. Operation ⓒ performs lookup on *cpus_count* map. Operation ⓓ updates the *destination IP address* of IPv4 header. Operation ⓔ reads *source port* from TCP header. Operation ⓕ writes *source port* to *store_sport* map.

byte instructions 16 and 17 calls map update helper function on *store_sport* map. For these instructions, we record the names of maps, fields written to map, and the helper functions associated with the *imm* value[4].

Rules R11 and R12 are applied to extract the list of protocol headers processed in the bytecode. Generally, while parsing packet data, a header is identified by looking at specific fields in the previously identified headers. For example, consider the first 14 bytes as an Ethernet header. Then, looking at the etherType field, we know the next header is a VLAN, an IPv4, or an IPv6 header. Similarly, looking at the ipv4.proto field, we know the next header is a TCP or a UDP header. We leverage the packet header semantics [11] and recognize headers. We also leverage that most bytecodes perform memory-bound checks before accessing the next header to ensure subsequent code accesses within the packet memory. If the next header is accessed without a memory-bound check, then the eBPF verifier throws an error and will not allow the loading of the bytecode. A running example bytecode with steps to illustrate this is in Figure 8 in Appendix A.2.

R13 retrieves the eBPF helper function invoked based on *imm* value, which represents an ID associated with each helper function. Helper function names are retrieved using the 'getHlprFuncName' support function. R14 captures the return action (*e.g.*, XDP_PASS, TC_ACT_OK) based on register r_0 value, which varies for different *hook* points[5](*e.g.*, XDP, TC). It also appends the current path's network context to C. Two maps, say A and B, are considered correlated if map A's lookup result is map B's key. Rule R15 retrieves the correlated maps based on register r_2 tag; it holds the key for map operations such as map update (call 2) and delete (call 3).

5 Language and Query Engine (QE)

Yaksha-Prashna Language. The grammar, as shown in Figure 1, follows a declarative (logical) program paradigm [75] with syntax similar to the widely used Prolog[6] language syntax [68, 77, 86]. The grammar exposes specialized eBPF-specific predicates that enable eBPF users, the primary target audience of the tool, to express assertions and queries easily by leveraging their existing knowledge of eBPF. The two key constructs are: (1) **Q-Predicates**: Q-Predicates (Q-Pred) are the primary building blocks of queries. They assert specific properties or retrieve network context. Q-Pred takes one or more arguments: when all arguments of a Q-Pred are defined with a specific value, the predicate asserts that the property holds. When one or more arguments are variables (*i.e.*, not defined), the Q-Pred retrieves all instances where the property applies, with variables being filled by the retrieved values, and (2) **Queries**: A query consists of one or more Q-Preds connected by logical AND(,)/OR(;) operator and ends with period (.).

5.1 Query Engine

Query Engine (QE) translates network context encoded in CFG-NC to prolog facts while maintaining the structural integrity of the CFG-NC and order of NFs in the NF chain. The user queries written using Yaksha-Prashna language (Figure 1) are answered by matching Q-Preds against the facts in the knowledge base. Table 2 illustrates Q-Predicates[7], prolog facts, and corresponding network context in CFG-NC. Facts are generated from the network context associated with each CFG-NC node. We call the complete set of facts for all bytecode(s) in an NF chain the knowledge base.

For mapping Q-Preds to facts, internally, we define rules that help in matching. The rules are of the form `Q-Pred`:- $\texttt{R-Pred}_1\, op\, \texttt{R-Pred}_2\, op\, \texttt{R-Pred}_3\, op \ldots, \texttt{R-Pred}_n$, where `Q-Pred` is the query predicate which constitutes the head of the rule, `R-Preds` correspond to the rule predicates constituting the rule body and *op* are the logical operators such as AND(,) and OR(;). As shown in Figure 5, rule heads are exposed to the operator (§1), and the body constitutes the operational

[4]'$\text{getMapName}_{\text{elf}}$()' does a look-up on ELF section of the object file and returns map's name.

[5]'getActionName' support function retrieves the action name.

[6]Prolog is just an implementation language used by Yaksha-Prashna; other logic programming languages could also be used.

[7]definitions are in Appendix A.4

Table 2. Predicates, network context, and associated facts.

Predicate Category	Network Context from CFG-NC	Facts
Field predicates	$C[read\_buff]$ = [(buff_field, field_name)], $C[read\_hdr]$ = [(hdr_field, field_name)], $C[write\_buff]$ = [(buff_field, field_name)], $C[write\_hdr]$ = [(hdr_field, field_name)]	read_buffer_field("nf_id", "bb_id", field_name), read_header_field("nf_id", "bb_id", field_name), write_buffer_field("nf_id", "bb_id", field_name), write_header_field("nf_id", "bb_id", field_name)
Map predicates	$C[read\_map]$ = [(map, map_name)], $C[write\_map]$ = [(map, map_name, field_name)] $C[correlated\_maps]$ = [(map, map_A, map_B)]	read_from_map("nf_id", "bb_id", map_name), write_into_map("nf_id", "bb_id", map_name, field_name) correlated_maps("nf_id", "bb_id", map_A, map_B)
Helper predicate	$C[helper]$ = [(helper, function_name)]	invoke_helper("nf_id", "bb_id", function_name)
Protocol predicate	$C[curr\_proto]$ = [(curr_proto, protocol_name)]	protocol_accessed("nf_id", "bb_id", protocol_name)
Action predicate	$C[pkt\_action]$ = [(hook, action, P)]	return_action("nf_id", hook, action, P)
Order predicates	–	edge("bb_id", "bb_id"), nf_edge("nf_id", "nf_id")

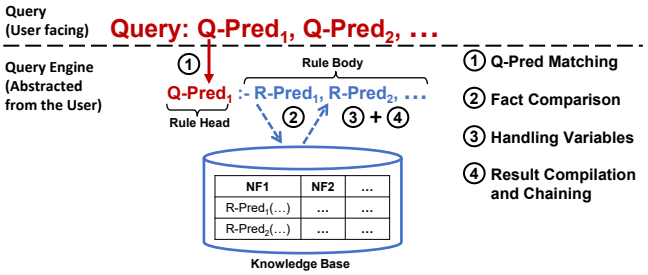


Figure 5. Query execution workflow.

semantics of the particular query predicate. This approach abstracts the details of the underlying facts while allowing users to write queries without needing to understand the detailed structure of the underlying facts. Table 5 in Appendix A.5 summarizes the operational semantics of all Q-Preds.

The following steps are involved in answering the user queries from the knowledge base. (1) **Q-Pred matching:** Each Q-Predicate in a query is compared against the heads of the rules defined. (2) **Fact comparison:** The R-Preds are matched against the facts stored in the knowledge base. (3) **Handling variables:** If the query contains variables, the engine retrieves all values that satisfy the rule from the knowledge base. (4) **Result compilation and chaining:** After the matching process, results are compiled by collecting all the values that satisfy the query predicates. Appendix A.6 presents a running example explaining these steps.

Yaksha-Prashna assertion execution. In §A.1, we explain the workings of Analyzer and Query Engine using three assertions, **A1**, **A2** and **A3** defined in Sec. §2.2, on two bytecodes (Listing 1 and 2 in Sec. §2.2).

Extensions. Yaksha-Prashna can be extended to support new user-defined Q-Predicates to match the facts. As discussed, the new Q-Predicates should have semantics with the corresponding R-Preds, and they can be incorporated as a library to support more queries for new use cases.

6 Evaluation

We evaluate Yaksha-Prashna to study: 1) Expressiveness of Yaksha-Prashna language, 2) Usage of Yaksha-Prashna (§6.2), 3) Accuracy of network context extraction (§6.3), 4) time to generate CFG-NC (§6.3) and to execute queries (§6.4), and 5) Memory overheads (§A.12).

Implementation. Yaksha-Prashna analyzer module consists of 1.8K LoC in C++. Internally, QE module invokes a SWI-Prolog (version 9.3.3) [86] environment, which takes facts and rules and provides a CLI for writing queries. This module consists of 600 LoC in C++. We ran experiments on a server with AMD EPYC 7262, 3.2 GHz, 8-core CPU, 32GB RAM.

6.1 Expressiveness

We evaluate language expressiveness by expressing properties and queries covering three categories §2.2 over half a dozen standard NFs, four non-standard but real eBPF-based NFs and NF chains. Table 3 lists the properties supported by Yaksha-Prashna, and closely related work Klint [70] and DRACO [61]. Each property is expressed in Yaksha-Prashna's language (Table 7), with a detailed explanation in Appendix §A.10. In this section, we summarize key observations.

The first category has 21 properties (P1-P21), as shown in Table 3, which are derived from the standards documents (RFCs and IEEE standards) for individual NFs. Yaksha-Prashna and Klint support most properties, whereas DRACO's support is unclear (–), except for Property P5 and Properties P16–P21, as it does not disclose the specification language. Properties P4 and P11 are partially supported by Yaksha-Prashna, because the language lacks predicates on map type and broadcast packets. The second category has properties on the NF chain to detect unintended interactions (§2.2). Currently, Klint supports assertions only on individual NFs, whereas Yaksha-Prashna and DRACO support assertions on dependencies in an NF chain.

The third category has retrieval queries (Q23 and Q24) that demonstrate the language's ability to support not only assertions, but also to retrieve specific details causing violations. More specifically, to find which field was modified by a firewall (Q23) that led to an assertion violation, Klint or DRACO would require writing a preservation assertion for each header field, such as *"Is the Source MAC modified?"* or *"Is the Source IP modified?"*, etc., which is tedious or infeasible as it requires checking every possible header and takes a long time for large programs. In comparison, Yaksha-Prashna can get this information instantly from the facts stored in the knowledge base. Similarly, Yaksha-Prashna can also retrieve the buffer fields (Q24), causing RAW dependency (Listing 2).

Table 3. Properties supported by Yaksha-Prashna, Klint and DRACO. (✗): partial support. Property's superscript has NF name – RT: Router, FW: Firewall, LB: Load balancer, NAT: NAT, BR: Bridge, POL: Policer, KTRN: Katran LB, CRAB: CRAB LB, FLU: Fluvia. ∗ represents all NFs

Category	Properties(P1-P22)/Retrieval Queries(Q23-Q24)	Yaksha-Prashna	Klint	DRACO
Category 1: Individual NF conformance to specification	P1. Expected header$^{(*)}$	✓	✓	-
	P2. Validity of IPv4 header$^{(RT)}$	✓	✓	-
	P3. Expected header field updates by NF$^{(RT)}$	✓	✓	-
	P4. Longest prefix matching$^{(RT)}$	✗	✓	-
	P5. Packet content preservation$^{(FW)(LB)}$	✓	✓	✓
	P6. Filtering external flows$^{(FW)(NAT)}$	✓	✓	-
	P7. Remembering internal flows$^{(FW)(NAT)}$	✓	✓	-
	P8. Update the source IP and port of the internal flow$^{(NAT)}$	✓	✓	-
	P9. Restore the destination IP and port of the external flow$^{(NAT)}$	✓	✓	-
	P10. Selected transmission port must not be the ingress port$^{(BR)}$	✓	✓	-
	P11. Broadcast if unknown$^{(BR)}$	✗	✓	-
	P12. The learning process$^{(BR)}$	✓	✓	-
	P13. External traffic policing$^{(POL)}$	✓	✓	-
	P14. Backend liveness$^{(LB)}$	✓	✓	-
	P15. Load balancing external traffic$^{(LB)}$	✓	✓	-
	P16. Drop all fragmented packets$^{(KTRN)}$	✓	✓	✓
	P17. Forward all ICMP echo request packets$^{(KTRN)}$	✓	✓	✓
	P18. Handle only TCP SYN packets$^{(CRAB)}$	✓	✓	✓
	P19. Add custom redirection header$^{(CRAB)}$	✓	✓	✓
	P20. Forward all packets unchanged$^{(FLU)}$	✓	✓	✓
	P21. Correlated maps$^{(hXDP-FW)}$	✓	✓	✓
Category 2: Unintended NF interactions	P22. NF dependency: Read after write (RAW), Write after read (WAR) and Write after write (WAW)	✓	✗	✓
Category 3: Understanding the NF/NF chain behavior	Q23. Find the list of packet fields updated by the NF	✓	✗	✗
	Q24. Find the list of buffer fields updated by one NF and read by successor NF (RAW dependent)	✓	✗	✗

6.2 Usage of Yaksha-Prashna

We demonstrate on a testbed server how an operator can use Yaksha-Prashna to check whether an NF bytecode conforms to its specification and if not, why it deviates from expected NF behavior (Category 1). Also, it captures unintended NF interactions (Category 2), and aids in understanding NF/NF-chain behavior (Category 3), as motivated in §2.2.

Category 1 use case. Consider that an NF operator wants to deploy a stateful firewall (Listing 1) with two bugs at the server's XDP hookpoint via bpfman (see Figure 9 in §A.8). The two bugs are: (1) updates the TCP source port (line 10), and (2) allows ICMP traffic (line 6). Before deploying, the operator uses Yaksha-Prashna to verify against two assertions, A1 and A2. A1 asserts that the firewall should not modify packet contents. Since the firewall modified the TCP source port, A1 failed, and Yaksha-Prashna raised an alert. To understand the reason, the operator runs a retrieval query RQ1 (Category 3), which correctly identifies the violation source as TCP source port update. Similarly, A2 asserts that ICMP traffic should be dropped. Since the firewall allowed ICMP packets, A2 failed, and Yaksha-Prashna raised another alert. Thus, through these assertions, Yaksha-Prashna successfully detected and prevented deployment of a non-standard NF.

Category 2 use case. We emulate a real-world RAW dependency between Cilium and the AWS Elastic Network Interface. An NF chain of three XDP-based NFs (Listing 2) was considered, with NF1 and NF3 deployed in sequence (NF1 → NF3). Before inserting NF2, the bytecodes of intended ordering (NF1 → NF2 → NF3) were verified using Assertion A3, which asserts that NF2's write set does not overlap

Table 4. Microbenchmark NFs. (*) NFs from open source projects, (+) Hand-written NFs, NI: # Instructions, NAI: #Instructions analyzed, and NP: # Paths.

ID	Network Functions	Project	NI	NAI	NP
NF1	xdp_pktcntr	Katran [64]	22	28	4
NF2	xdp_map_access	K2 [88]	42	64	6
NF3	storeIPinMap+	–	39	49	7
NF4	xdp_fw	K2 [88]	73	496	38
NF5	ratelimiting_kern_new	ebpf-ratelimiter [14]	136	292	19
NF6	ratelimiting_kern	ebpf-ratelimiter [14]	167	278	18
NF7	xdp1_kern	K2 [88]	61	925	99
NF8	decap_kern	Katran [64]	228	430	33
NF9	xdp_filter	Suricata [35]	344	13K	639
NF10	xdp_filter_buff_update*	Suricata [35]	348	13K	639
NF11	mptm_encap_xdp	Mptm [15]	348	25K	1K
NF12	mptm_decap_xdp	Mptm [15]	149	38K	2K
NF13	xdp_lb	Suricata [35]	419	65K	4K
NF14	xdp_lb_hdr_update*	Suricata [35]	423	65K	4K
NF15	xdp_lb_syn_storeIPinMap*	Suricata [35]	440	66K	4K
NF16	xdp_lb_syn_storeIPTTLinMap*	Suricata [35]	460	68K	4K

with NF3's read set. Assertion A3 failed due to an overlap on *sk_buff->mark*, and Yaksha-Prashna raised an alert. Next, using retrieval query RQ3 (Category 3), we pinpoint the cause of the violation. Hence, Yaksha-Prashna prevented an unintended interaction that would have occurred upon deployment of a new NF into the existing NF chain.

6.3 Network context extraction

NFs and Queries. We generate bytecodes using *LLVM Clang* (V11.1.0) with O2, Oz, and O3 flags for a microbenchmark suite of sixteen XDP-based NFs (Table 4). The selected NFs represent a diverse set of real-world XDP programs drawn from widely used open-source and production systems. They cover common packet-processing tasks, including packet counting, map access, firewalling, rate limiting, and load balancing. The suite spans a broad range of complexities and instruction counts: simple NFs (e.g., NF1 and NF2) implement basic functionality such as packet counting with 22–42 instructions, whereas more complex NFs (e.g., NF4, NF9, and NF13) perform firewalling, filtering, and load balancing with several hundred instructions, with the largest exceeding 400. This diversity enables us to evaluate our system across both lightweight and compute-intensive NFs representative of practical deployments. We design a total of 16 queries (Table 6 in Appendix A.7) and assess the structural properties of individual NFs and the interactions across multiple NFs in an NF-Chain. These queries are evenly divided, with 8 focusing on assertions and 8 on retrieval, covering all predicate and rule categories detailed in Table 2 and Table 5.

Accuracy. We define accuracy as the proportion of network context information extracted by the analyzer compared to the information in the ground truth. To establish the ground truth, we manually review the source code to extract list of protocols, specific fields accessed in each protocol/buffers, helper functions, and maps. The analyzer extracts information correctly for all NFs, except for some fields in NF12. We also compare the assertion query results (*i.e.*, Q1(A)–Q8(A)) for randomly stacked facts of microbenchmark NFs (Table 4) and found the answers are correct.

Our system assumes that an NF performs a next-protocol check followed by a memory-bound check before accessing protocol header fields. This assumption is derived from real-world implementations and aligns with common programming conventions. The next-protocol check enables the NF to apply protocol-specific processing logic, while the memory-bound check ensures that header accesses remain within packet bounds, which is necessary to pass the verifier. In practice, developers structure packet-parsing logic in this manner, and similar patterns can be observed in production systems such as Katran and Suricata.

Although this assumption holds for most NFs, we observe exceptions. In NF12 (mptm_decap_xdp), our system is unable to identify the UDP protocol and its associated header fields. This is because the NF does not perform an explicit next-protocol check for UDP; instead, it treats all non-TCP traffic as UDP by default.

When an NF is designed to operate on only a subset of traffic (e.g., TCP and UDP) and does not implement comprehensive protocol checks, our system may fail to accurately infer the intended protocol semantics. This behavior highlights the limitation of the current version of our system. However this limitation could be addressed by extending our system to handle NFs that are tailored to restricted traffic classes rather than fully general packet processing.

CFG-NC generation time. We evaluated the time taken by our system to generate CFG-NC across various NFs, as shown in Figure 10 (left) in §A.9. The results are averaged over five runs and show a clear relationship between CFG-NC generation time and complexity of NFs. We observed that the time to generate a CFG-NC scales with the number of paths in the CFG. For NFs with 4 to 4K paths, the generation time ranges from approximately 3 to 300 milliseconds. For NFs like NF13–NF16, which have the same number of paths but different numbers of instructions, the time proportionally increases with the increase in the number of instructions. We observe no significant difference in the time to extract network context for O3-optimized bytecode and other optimization flags.

6.4 Query execution

The Query Engine is built on the highly optimized swi-prolog runtime (V7.6.4) [86][8]; all of our query execution takes a few microseconds. We analyze the query execution time under

[8] swi-prolog implements several optimizations like indexing and unification to optimize the query resolution

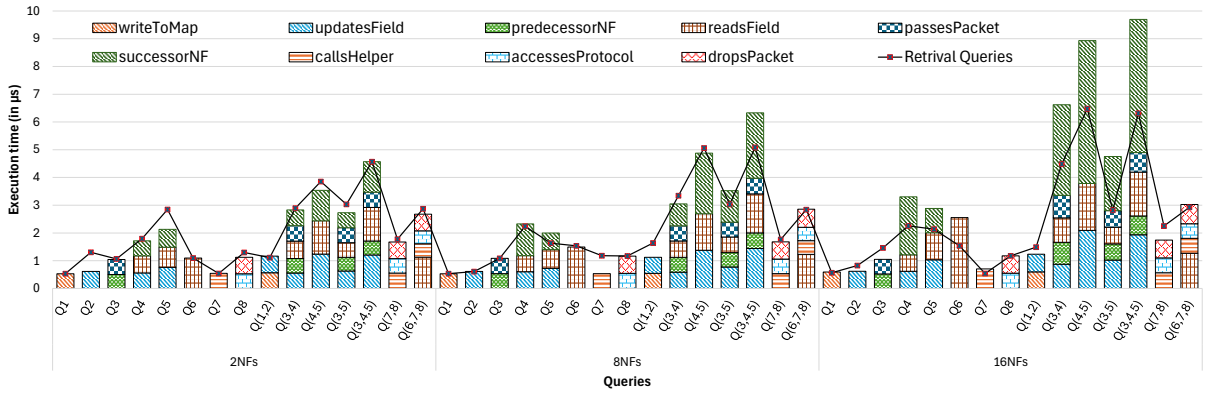


Figure 6. Execution time: assertions (bars) and retrieval queries (line) on NF chains of varying length, where NFs are chosen randomly.

different scenarios involving (i) assertion and retrieval, (ii) presence of recursive predicates, (iii) different numbers of Q-Predicates (§A.11), and (iv) varying lengths of NF-chain (§A.11). To reduce the impact of noise, we run each query on the microbenchmarks 1000 times, obtain a median for every 100 runs, and then average the medians.

Retrieval and Assertion queries. The execution time for each Q-Predicates in the retrieval and assertion queries are shown in Figure 6. We observe that the individual retrieval queries Q1(R)–Q8(R) and assertion queries Q1(A)–Q8(A) took 4μs when executed on the facts obtained from 16 NFs. The combination variants of retrieval and assertion queries took 1.1μs–6.4μs and 1.1μs–9.6μs, respectively.

Recursive predicates. A rule is said to be recursive when the rule body includes a predicate that refers to the same rule head. For example, the predicates `successorNF` and `predecessorNF` shown in Table 5 are recursive predicates. We observe that the queries involving such recursive predicates take more time to execute than the non-recursive ones due to the additional overhead of handling recursive calls. These queries, when executed on the facts of sixteen NFs, took 1.04μs–3.3μs in comparison with the execution time of queries with no recursive predicates (0.5μs–2.5μs).

Comparison with Klint. Figure 7 (left) shows verification time of Yaksha-Prashna and Klint on six representative NFs for 15 properties: P1–P15 (Appendix §A.10.1). Yaksha-Prashna verification time includes CFG-NC generation and assertion execution, while for Klint it covers symbolic execution, invariant inference, and verification. Klint took 2.7 seconds to 1.5 minutes, whereas Yaksha-Prashna finishes in 13-75 ms, achieving a 200-1000× speedup due to its lightweight dataflow analysis compared to Klint's symbolic execution.

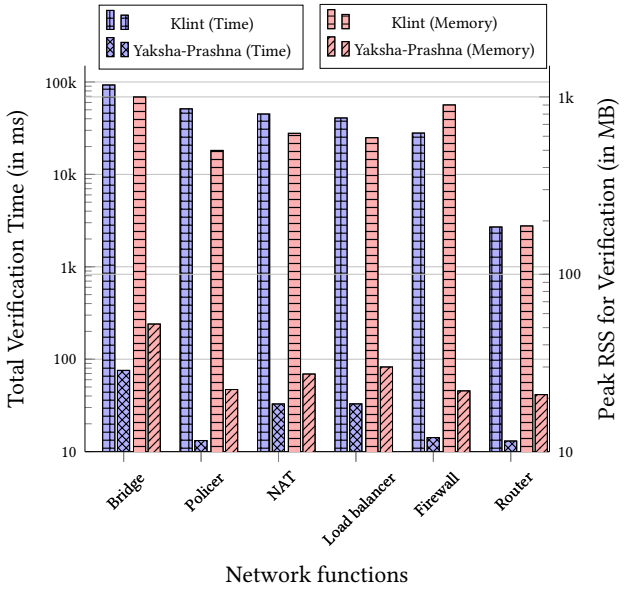


Figure 7. Verification time (left) and Memory footprint (right).

7 Limitations

In this section, we discuss the limitations of Yaksha-Prashna. Section §A.3 in Appendix has more discussion points on identifying the packet-processing algorithms, complexity with random guesses for identifying network context information, and possible extensions to the language and the analyzer.

Runtime queries. NFs often determine packet actions based on header field values. For example, a load balancer uses flow information to forward subsequent packets to the same destination. The Yaksha-Prashna does not support verification of such runtime properties, as they require runtime information, which is unavailable during the static analysis phase.

Compliance assertions on map rules. Maps are a key part of eBPF programs, but incorrect rules or missing actions can cause unexpected packet behavior. Operators often want to check the compliance of the map rules (e.g., absence of wildcard rules [58]), before insertion. Assuming map rules are given, Yaksha-Prashna does not support these assertions.

Support for additional hookpoints. Network functions can be attached to several hookpoints [5] such as XDP, TC, and socket filters. Currently, Yaksha-Prashna supports only XDP programs. Extending the tool to support additional hookpoints is primarily an engineering effort rather than a

fundamental limitation of our approach, as differences across hooks (e.g., execution context structures, field offsets, and return codes) can be incorporated with additional implementation work.

8 Related work

Static analysis for verification. Widely used eBPF verifiers like Linux eBPF verifier [82] and Prevail [40] use static analysis and abstract interpretation [27] to validate program termination, memory safety, and resource boundedness to ensure the kernel safety, but not the network context of the eBPF bytecode. Bromberger et al. [21] propose verification of the functional properties of the eBPF program using Z3 SMT solver [30]. Previous works have also used program verification for identifying bugs in P4 [60, 78]. Gravel [91], Vigor [90], and VEP [87] employ symbolic execution for verifying non eBPF-based NFs. Beyond finding dependencies, Yaksha-Prashna can be extended to verify network policies for stateful networks as in NetSMC [89]. ASSERT-P4 [38] and DRACO [61] are Klee symbolic execution [23] based approaches to verify P4/C programs.

Static analysis for optimizations. Merlin [62], KFuse [54], K2 [88], and Bonola et al. [19] propose static analysis based approaches for optimizing eBPF bytecode. Merlin performs eBPF aware optimizations on LLVM-IR, while KFuse and K2 design optimizations on the verified eBPF bytecode. Morpheus [66] proposes runtime optimizations on eBPF programs based on the profiles of map access patterns. On the other hand, our system focuses on retrieving the network contexts of the eBPF bytecode.

DSLs. Domain-specific languages (DSLs) like Halide [71] (for image processing) and NetBlocks [20] (for ad-hoc network protocol design) have helped create transpiler systems to obtain performance on the specific domains they target. Yaksha-Prashna language is inspired by other DSLs like Datalog [25] and NDlog [59] that are based on the declarative language paradigm. These languages are designed to query and reason databases and networked systems in a distributed environment. On the other hand, our language is designed to query and reason about the NF behavior of eBPF bytecodes.

9 Conclusion

We present Yaksha-Prashna, a system for understanding eBPF-based NF bytecode and its interaction with other NFs. We propose (1) Analyzer to extract and model network context using dataflow analysis and propagation; and (2) Language, a simple but expressive domain-specific language for NF operators/developers to assert and retrieve eBPF bytecode behavior while abstracting out the complex low-level bytecode details. We evaluate (1) Yaksha-Prashna language expressiveness by writing a wide range of queries checking NF properties, (2) Yaksha-Prashna performance for 16 XDP programs, and (3) compared Yaksha-Prashna with state-of-the-art tools, Klint and DRACO.

References

[1] 2024. eBPF Instruction Set Specification, v1.0. https://docs.kernel.org/6.5/bpf/instruction-set.html.

[2] 2024. Fluvia source code respository. https://github.com/nttcom/fluvia.

[3] 2024. Linux manual page. https://www.man7.org/linux/man-pages/man1/time.1.html.

[4] 2025. bpfman: An eBPF Manager. https://bpfman.io/v0.5.6/.

[5] 2025. Categorization of eBPF Hooks and Use Cases. https://eunomia.dev/others/miscellaneous/ebpf-usecases/.

[6] 2025. Equinix Network Edge. https://www.equinix.com/products/digital-infrastructure-services/network-edge.

[7] 2025. F5 NGINX App Protect. https://www.f5.com/products/nginx/nginx-app-protect.

[8] 2025. libxdp. https://github.com/xdp-project/xdp-tools/tree/main/lib/libxdp.

[9] 2025. NGINX App Protect DoS 3.0. https://docs.nginx.com/nginx-app-protect-dos/releases/about-3.0/#important-notes.

[10] 2025. PAN-OS Release Notes. https://docs.paloaltonetworks.com/content/dam/techdocs/en_US/pdf/pan-os/10-1/pan-os-release-notes/pan-os-release-notes.pdf.

[11] The PANDA Authors. 2023. PANDA (Protocol And Network Datapath Acceleration). https://github.com/panda-net/panda/blob/main/documentation/images/Parse-graph.png.

[12] Ryan Beckett, Xuan Kelvin Zou, Shuyuan Zhang, Sharad Malik, Jennifer Rexford, and David Walker. 2014. An assertion language for debugging SDN applications. In *HotSDN*.

[13] Dushyant Behl, Hai Huang, Palani Kodeswaran, and Sayandeep Sen. 2023. On eBPF extensions to Kubernetes CNI datapath. In *COMSNETS*.

[14] Dushyant Behl, Sayandeep Sen, and Palani Kodeswaran. 2023. https://github.com/ebpf-networking/ebpf-ratelimiter.

[15] Dushyant Behl, Sayandeep Sen, and Palani Kodeswaran. 2023. xdp-mptm. https://github.com/dushyantbehl/xdp-mptm.

[16] Theophilus A. Benson, Prashanth Kannan, Prankur Gupta, Balasubramanian Madhavan, Kumar Saurabh Arora, Jie Meng, Martin Lau, Abhishek Dhamija, Rajiv Krishnamurthy, Srikanth Sundaresan, Neil Spring, and Ying Zhang. 2024. NetEdit: An Orchestration Platform for eBPF Network Functions at Scale. In *ACM SIGCOMM*.

[17] Gilberto Bertin. 2017. XDP in practice: integrating XDP into our DDoS mitigation pipeline. In *Technical Conference on Linux Networking, Netdev*.

[18] Brenden Blanco, Jakub Kicinski, Salvatore Orlando, and Tomás Senart. 2023. Cilium. https://github.com/cilium/cilium.

[19] Marco Bonola, Giacomo Belocchi, Angelo Tulumello, Marco Spaziani Brunella, Giuseppe Siracusano, Giuseppe Bianchi, and Roberto Bifulco. 2022. Faster Software Packet Processing on {FPGA}{NICs} with {eBPF} Program Warping. In *USENIX ATC*.

[20] Ajay Brahmakshatriya, Chris Rinard, Manya Ghobadi, and Saman Amarasinghe. 2024. NetBlocks: Staging Layouts for High-Performance Custom Host Network Stacks. (2024).

[21] Martin Bromberger, Simon Schwarz, and Christoph Weidenbach. 2024. Automatic Bit-and Memory-Precise Verification of eBPF Code. In *Proceedings of 25th Conference on Logic for Pro*.

[22] Marco Spaziani Brunella, Giacomo Belocchi, Marco Bonola, Salvatore Pontarelli, Giuseppe Siracusano, Giuseppe Bianchi, Aniello Cammarano, Alessandro Palumbo, Luca Petrucci, and Roberto Bifulco. 2020. hXDP: Efficient Software Packet Processing on FPGA NICs. In *USENIX OSDI*.

[23] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for

Complex Systems Programs. In *USENIX OSDI 08*.

[24] Carlos Castro. 2024. Traffic dropped for identity not found. https://github.com/cilium/cilium/issues/20797/#issuecomment-1238841882.

[25] S. Ceri, G. Gottlob, and L. Tanca. 1989. What you always wanted to know about Datalog (and never dared to ask). *IEEE Transactions on Knowledge and Data Engineering* (1989).

[26] Yean-Ru Chen, Si-Han Chen, and Shang-Wei Lin. 2023. SMT Solver With Hardware Acceleration. *Trans. Comp.-Aided Des. Integ. Cir. Sys.* (2023).

[27] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*.

[28] Patrick Cousot and Nicolas Halbwachs. 1978. Automatic discovery of linear restraints among variables of a program. In *POPL*.

[29] Datadog. 2024. Network Performance Monitoring. https://www.datadoghq.com/product/network-monitoring/network-performance-monitoring/.

[30] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *TACAS*.

[31] Mugdha Deokar, Jingyang Men, Lucas Castanheira, Ayush Bhardwaj, and Theophilus A. Benson. 2024. An Empirical Study on the Challenges of eBPF Application Development. In *ACM SIGCOMM eBPF and Kernel Extensions workshop*.

[32] Alan AA Donovan and Brian W Kernighan. 2015. *The Go programming language*. Addison-Wesley Professional.

[33] Robert W Floyd. 1993. Assigning meanings to programs. In *Program Verification: Fundamental Issues in Computer Science*. Springer.

[34] Stanislav Fomichev, Eric Dumazet, Willem de Bruijn, Vlad Dumitrescu, Bill Sommerfeld, and Peter Oskolkov. 2024. Replacing HTB with EDT and BPF. https://netdevconf.info//0x14/pub/papers/55/0x14-paper55-talk-paper.pdf.

[35] Open Information Security Foundation. 2023. Suricata. https://github.com/OISF/suricata.

[36] Guillaume Fournier and Hemanth Malla. 2023. Tales from an eBPF Program's Murder Mystery. https://www.youtube.com/watch?v=YK7GyEJdJGo.

[37] Guillaume Fournier and Hemanth Malla. 2023. Troubles and Tidbits from Datadog's eBPF Journey. https://lpc.events/event/17/contributions/1582/attachments/1328/2673/Troubles%20and%20Tidbits%20from%20Datadog%E2%80%99s%20eBPF%20Journey%20v2.pdf.

[38] Lucas Freire, Miguel Neves, Lucas Leal, Kirill Levchenko, Alberto Schaeffer-Filho, and Marinho Barcellos. 2018. Uncovering Bugs in P4 Programs with Assertion-based Verification. In *SOSR*.

[39] Michael R Garey and David S Johnson. 1979. *Computers and intractability*. Vol. 174. freeman San Francisco.

[40] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted linux kernel extensions. In *PLDI*.

[41] Rachael Graham. 2024. IBM Cloud Kubernetes Service now provides sets of Calico network policies to isolate your cluster on public and private networks. https://www.ibm.com/blog/isolating-clusters-with-calico-policies-in-ibm-cloud-kubernetes-service/.

[42] D. Gries. 1981. *The Science of Programming*. Springer New York. https://books.google.co.in/books?id=5fUYAQAAIAAJ

[43] Matthew S. Hecht. 1977. *Flow Analysis of Computer Programs*. Elsevier Science Inc.

[44] C. A. R. Hoare. 1969. An axiomatic basis for computer programming. *Commun. ACM* (1969).

[45] Alfred Horn. 1951. On Sentences Which Are True of Direct Unions of Algebras. *Journal of Symbolic Logic* (1951).

[46] Roberto Ierusalimschy. 2006. *Programming in lua*. Roberto Ierusalimschy.

[47] Rishabh Iyer, Katerina Argyraki, and George Candea. 2022. Performance Interfaces for Network Functions. In *USENIX NSDI*.

[48] John B. Kam and Jeffrey D. Ullman. 1976. Global Data Flow Analysis and Iterative Algorithms. *J. ACM* (1976).

[49] John B Kam and Jeffrey D Ullman. 1977. Monotone data flow analysis frameworks. *Acta informatica* (1977).

[50] The Linux kernel Authors. 2024. Classic BPF vs eBPF. https://docs.kernel.org/bpf/classic_vs_extended.html.

[51] Brian W Kernighan and Dennis M Ritchie. 2002. The C programming language. (2002).

[52] Gary A. Kildall. 1973. A unified approach to global program optimization. In *POPL*.

[53] Marios Kogias, Rishabh Iyer, and Edouard Bugnion. 2020. Bypassing the load balancer without regrets. In *Proceedings of the 11th ACM Symposium on Cloud Computing*.

[54] Hsuan-Chi Kuo, Kai-Hsun Chen, Yicheng Lu, Dan Williams, Sibin Mohan, and Tianyin Xu. 2022. Verified programs can party: optimizing kernel extensions via post-verification merging. In *EuroSys*.

[55] l3af project. 2023. eBPF Package Repository. https://github.com/l3af-project/eBPF-Package-Repository.

[56] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *IEEE CGO*.

[57] Guyue Liu, Hugo Sadok, Anne Kohlbrenner, Bryan Parno, Vyas Sekar, and Justine Sherry. 2021. Don't Yank My Chain: Auditable NF Service Chaining. In *USENIX NSDI*.

[58] Jed Liu, William Hallahan, Cole Schlesinger, Milad Sharif, Jeongkeun Lee, Robert Soulé, Han Wang, Călin Caşcaval, Nick McKeown, and Nate Foster. 2018. p4v: practical verification for programmable data planes. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*.

[59] Boon Thau Loo, Tyson Condie, Minos Garofalakis, David E. Gay, Joseph M. Hellerstein, Petros Maniatis, Raghu Ramakrishnan, Timothy Roscoe, and Ion Stoica. 2009. Declarative networking. *Commun. ACM* (2009).

[60] Nuno Lopes, Nikolaj Bjørner, Nick McKeown, Andrey Rybalchenko, Dan Talayco, and George Varghese. 2016. Automatically verifying reachability and well-formedness in P4 Networks. *Technical Report, Tech. Rep* (2016).

[61] Dana Lu, Boxuan Tang, Michael Paper, and Marios Kogias. 2024. Towards Functional Verification of eBPF Programs. In *ACM SIGCOMM eBPF and Kernel Extensions workshop*.

[62] Jinsong Mao, Hailun Ding, Juan Zhai, and Shiqing Ma. 2024. Merlin: Multi-tier Optimization of eBPF Code for Performance and Compactness. In *ASPLOS*.

[63] Nicholas D Matsakis and Felix S Klock. 2014. The rust language. *ACM SIGAda Ada Letters* (2014).

[64] Meta. 2023. katran. https://github.com/facebookincubator/katran.

[65] Meta. 2024. Open-sourcing Katran, a scalable network load balancer. https://engineering.fb.com/2018/05/22/open-source/open-sourcing-katran-a-scalable-network-load-balancer/.

[66] Sebastiano Miano, Alireza Sanaee, Fulvio Risso, Gábor Rétvári, and Gianni Antichi. 2022. Domain specific run time optimization for software data planes. In *ASPLOS*.

[67] Miguel Neves, Lucas Freire, Alberto Schaeffer-Filho, and Marinho Barcellos. 2018. Verification of P4 programs in feasible time using assertions. In *ACM CoNEXT*.

[68] Richard O'Keefe. 2009. *The craft of Prolog*. MIT press.

[69] Gergely Orosz. 2023. Inside Datadog's $5M Outage (Real-World Engineering Challenges #9). https://newsletter.pragmaticengineer.com/p/inside-the-datadog-outage.

[70] Solal Pirelli, Akvilė Valentukonytė, Katerina Argyraki, and George Candea. 2022. Automated Verification of Network Function Binaries. In *USENIX NSDI*.

[71] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *PLDI*.
[72] H. G. Rice. 1953. Classes of Recursively Enumerable Sets and Their Decision Problems. *Trans. Amer. Math. Soc.* (1953).
[73] J. A. Robinson. 1965. A Machine-Oriented Logic Based on the Resolution Principle. *J. ACM* (1965).
[74] Michel F Sanner et al. 1999. Python: a programming language for software integration and development. *J Mol Graph Model* (1999).
[75] Michael Scott. 2000. *Programming language pragmatics*. Morgan Kaufmann.
[76] Alexei Starovoitov. 2024. eBPF Syscall. https://docs.kernel.org/userspace-api/ebpf/syscall.html.
[77] Leon Sterling and Ehud Y Shapiro. 1994. *The art of Prolog: advanced programming techniques*. MIT press.
[78] Radu Stoenescu, Dragos Dumitrescu, Matei Popovici, Lorina Negreanu, and Costin Raiciu. 2018. Debugging P4 programs with Vera. In *ACM SIGCOMM*.
[79] Bjarne Stroustrup. 1994. *The design and evolution of C++*. Pearson Education India.
[80] Bjarne Stroustrup. 2022. *A Tour of C++*. Addison-Wesley Professional.
[81] Bingchuan Tian, Jiaqi Gao, Mengqi Liu, Ennan Zhai, Yanqing Chen, Yu Zhou, Li Dai, Feng Yan, Mengjing Ma, Ming Tang, Jie Lu, Xionglie Wei, Hongqiang Harry Liu, Ming Zhang, Chen Tian, and Minlan Yu. 2021. Aquila: a practically usable verification system for production-scale programmable data planes. In *ACM SIGCOMM*.
[82] Linus Trovalds. 2024. Verifier. https://github.com/torvalds/linux/blob/de927f6c0b07d9e698416c5b287c521b07694cac/Documentation/bpf/verifier.rst.
[83] S VenkataKeerthy, Yashas Andaluri, Sayan Dey, Rinku Shah, Praveen Tammana, and Ramakrishna Upadrasta. 2022. Packet Processing Algorithm Identification using Program Embeddings. In *APNET*.
[84] Marcos AM Vieira, Matheus S Castanho, Racyus DG Pacífico, Elerson RS Santos, Eduardo PM Câmara Júnior, and Luiz FM Vieira. 2020. Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications. *ACM Computing Surveys (CSUR)* (2020).
[85] Jan Wielemaker. 2009. *Logic programming for knowledge-intensive interactive applications*. Ph. D. Dissertation. University of Amsterdam. http://dare.uva.nl/en/record/300739.
[86] Jan Wielemaker, Tom Schrijvers, Markus Triska, and Torbjörn Lager. 2012. Swi-prolog. *Theory and Practice of Logic Programming* (2012).
[87] Xiwei Wu, Yueyang Feng, Tianyi Huang, Xiaoyang Lu, Shengkai Lin, Lihan Xie, Shizhen Zhao, and Qinxiang Cao. 2025. VEP: A Two-stage Verification Toolchain for Full eBPF Programmability. In *USENIX NSDI*.
[88] Qiongwen Xu, Michael D Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. 2021. Synthesizing safe and efficient kernel extensions for packet processing. In *ACM SIGCOMM*.
[89] Yifei Yuan, Soo-Jin Moon, Sahil Uppal, Limin Jia, and Vyas Sekar. 2020. NetSMC: A Custom Symbolic Model Checker for Stateful Network Verification. In *NSDI 20*.
[90] Arseniy Zaostrovnykh, Solal Pirelli, Rishabh Iyer, Matteo Rizzo, Luis Pedrosa, Katerina Argyraki, and George Candea. 2019. Verifying software network functions with no verification expertise. In *SOSP*.
[91] Kaiyuan Zhang, Danyang Zhuo, Aditya Akella, Arvind Krishnamurthy, and Xi Wang. 2020. Automated Verification of Customizable Middlebox Properties with Gravel. In *USENIX NSDI*.

A Appendix

A.1 Yaksha-Prashna Assertion Execution

In this section, we explain the workings of Analyzer and Query Engine using three assertions (**A1**, **A2** and **A3** defined in Sec. §2.2) executed on two bytecodes (Listing 1 and 2 in Sec. §2.2).

Consider assertions **A1** and **A2** on the firewall bytecode (Listing 1). The analyzer builds bytecode's CFG and extracts network context by applying transfer function rules in Table 1. More specifically, Rule **R4** extracts the fields updated by the bytecode, Rules **R11** and **R12** extract the list of protocols accessed, and Rule **R14** gathers the network context of all CFG nodes in each path. This information is stored in the knowledge base while maintaining the control flow structure in the bytecode. The Query Engine (QE) then executes the assertions: for **A1**, *updatesField* retrieves the updated fields (*i.e.*, TCP source port), causing the assertion to fail as the list is non-empty; for **A2**, *passes* gathers the network context on all paths with XDP_PASS as return action, and finally retrieves the list of protocols (*i.e.*, TCP, UDP, and ICMP) processed in these paths. The assertion fails because ICMP is in the list.

Next consider assertion **A3** on a chain of bytecodes, NF1 → NF2 → NF3, as shown in Listing 2. We assume the operator provides the order of NFs in a new or old chain. For instance, in the bpfman tool [4], NF priorities at a hook point define the order of their execution. Transfer function rules (Table 1) **R3** and **R4** extract the write list, that is, the list of fields updated by the bytecode, and the rules **R8** and **R9** extract the read list, the list of fields read by the bytecode. After applying transfer function rules to the bytecode, the extracted network context is stored in the knowledge base. While executing **A3** on the knowledge base, QE finds NF2's write list (*i.e.*, `skb->mark` field) overlaps with the read list of NF3, which indicates dependency between NF2 and NF3. This understanding of the dependencies on the yet-to-be-deployed NF chain helps operators to find unexpected interactions and prevent outages before they occur.

Using the bpfman tool [4], we deployed NF1 and NF3 at the XDP hook point, and executed assertion **A3** before deploying NF2. Figure 9 in Appendix A.8 illustrates the deployment of an example bytecode in the middle of an NF chain at a virtual interface.

A.2 Header identification

We apply rules R11 and R12 in Table 1 to find instructions that do header identification and memory-bound checks before accessing a header, respectively. More specifically, in R11, if the register points to a field[9] that identifies the

[9] isTailField is a supporting function on spec that check whether the destination register refers to the current protocol's header field that identifies the next header.

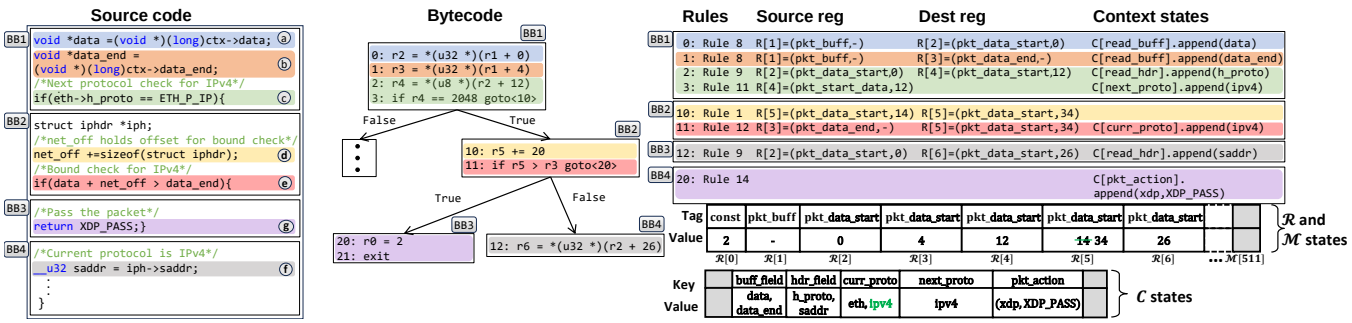


Figure 8. Header identification using specific fields in the previously identified headers.

next header (*e.g.*, ipv4.proto), then the next header's protocol is identified based on the immediate value (*e.g.*, 6 for TCP). A running example that illustrates these steps is shown in Figure 8. Here, we retrieve the next header's protocol using the 'getProtoName$_{spec}$' support function and store it in C with key as **next_proto**. Through R12, we confirm that the bytecode accesses the next header by ensuring that instructions associated with a memory-bound check are present. If present, we update the curr_proto with the next_proto. However, bytecode instructions that identify headers and perform memory-bound checks are spread across multiple basic blocks, because if a memory-bound check fails, the packet processing is halted. This requires forwarding protocol context (*i.e.*, next_proto and curr_proto) to successor basic blocks, which we do through state propagation, as mentioned earlier.

A.3 Discussion

Runtime inputs. Yaksha-Prashna is designed to run queries on facts representing the network context of eBPF bytecode. Yaksha-Prashna's analyzer and language can be extended to understand or assert the packet-processing behavior at runtime for a specific packet header with a set of key-value entries in maps. For example, *"All packets from a flow should be forwarded to the same destination"*. This helps to (1) debug connectivity and performance issues at runtime and (2) distinguish whether the unexpected packet-processing behavior (e.g., drop) is because of bugs in bytecode or an incorrect match rule in a map.

Identifying packet-processing algorithms. Currently, Yaksha-Prashna analyzer extracts header or metadata fields accessed or updated by tracking packet buffer offsets. Due to the limitations of the static-analysis techniques [72], we are limited to scenarios that can be expressed as decidable rules. It can be extended for identifying packet-processing algorithms (*e.g.*, hash, DDoS detection, load balance, encryption) powered by ML techniques on the bytecode. This enables NF audit that ensures the bytecode is processing packets as expected [57] or provides opportunities to offload to accelerators [83].

Query engine. Due to the reliance on the `swi-prolog` runtime, Yaksha-Prashna benefits from the decades of research on optimizations on query resolutions in logical paradigm languages [45, 73, 85]. This results in a highly scalable and lightweight runtime system, where query execution barely takes a few microseconds with a small memory footprint.

Comparison with random guess. Identifying the network context becomes increasingly complex with random guesses due to the combinatorial space involved. Consider a bytecode with p protocols with f fields interacting with m maps and h helper functions. The user has to guess among $\binom{p_{max}}{p} \times \binom{f_{max}}{f} + \binom{m_{max}}{m} + \binom{h_{max}}{h}$ possible combinations. For instance, consider NF9 with $(p = 6, f = 10, h = 2, m = 9)$ and $(p_{max} = 20, f_{max} = 15, m_{max} = 10, h_{max} = 50)$, which amount to $\binom{20}{6} \times \binom{15}{10} + \binom{10}{2} + \binom{50}{9} \approx 2.6 \times 10^9$ combinations. If the network context is unknown, the problem scales exponentially as the search space expands to $(2^{p_{max}-1} + 1) \times (2^{f_{max}-1} + 1) + (2^{m_{max}-1} + 1) + (2^{h_{max}-1} + 1) \approx 5.6 \times 10^{14}$ combinations (eBPF has 211 helper functions, which amount to 2×10^{15} and 1.64×10^{63} combinations when (p, f, m, h) are known and unknown, respectively). This emphasizes the challenges of identifying network context and the difficulty of random guessing, given the vast number of possibilities. By leveraging Yaksha-Prashna, the time and effort can be significantly reduced, a big advantage over random guessing.

A.4 Yaksha-Prashna language predicates

1. **Field predicates** *(field_pred)*: Check if an NF reads or writes to a specific header or buffer field. The predicate returns the list of all fields read or updated if the field argument is a variable.
2. **Map predicates** *(map_operation_pred, correlated_map_pred):* The former predicate checks operations on maps, and the latter predicate checks for a possible correlation between the maps (*e.g.*, map A lookup result is map B's key).
3. **Action predicates** *(pkt_act_pred):* Check if an NF performs actions like drop or redirect packets of a specific protocol. If protocol details are not mentioned, it returns the list of program paths (P) and the associated network context.
4. **Helper predicates** *(helper_pred):* Checks if an NF invokes a specific helper function.

5. **Protocol predicates** *(protocol_pred):* Checks if an NF processes a specific protocol.
6. **Order predicates** *(order_pred):* This predicate returns predecessor or successor NFs in an NF chain.

A.5 Operational Semantics of Yaksha-Prashna

Table 5. Operational Semantics of the Yaksha-Prashna.

Rule Head	Rule Body
`readsField(Nf, Fld):-`	`read_header_field(Nf, _, Fld); read_buffer_field(Nf, _, Fld).`
`updatesField(Nf, Fld):-`	`write_header_field(Nf, _, Fld); write_buffer_field(Nf, _, Fld).`
`successorNF(Nf, SNf):-`	`edge(Nf, SNf);edge(Nf, IntNf), successorNF(IntNf, SNf).`
`predecessorNF(Nf, PNf):-`	`edge(PNf, Nf); edge(PNf, IntNf), predecessorNF(Nf, IntNf).`
`passes(Nf, Hook, [(Fld, Val)]):-`	`return_action(Nf, Hook,"XDP_PASS", [(Fld, val)]`
`drops(Nf, Hook, [(Fld, Val)]):-`	`return_action(Nf, Hook,"XDP_DROP", [(Fld, val)]`
`mapWrite(Nf, Map, Fld):-`	`write_into_map(Nf, _, Fld, Map).`
`mapLookup(Nf, Map):-`	`read_from_map(Nf, _, Map).`
`correlatedMaps(Nf, MapA, MapB):-`	`correlated_map(Nf, _, MapA, MapB).`
`accessesProtocol(Nf, Fld, Val):-`	`protocol_accessed(Nf, _, Fld, Val).`
`callsHelper(Nf, Helper):-`	`invoked_helper(Nf, _, Helper).`

A.6 Query execution workflow

We explain the steps in answering the queries using `updatesField(Nf, Fld)` query and associated facts in the knowledge base.

Rule: `updatesField(Nf, Fld):- write_header_field(Nf, _, Fld).`

Fact: `write_header_field("firewall_kern_kern/xdp", node_3, ipv4.dst).`

Q-Pred matching: Each Q-Predicate in a query is compared against the heads of the rules defined. The Query Engine evaluates the body of the rule corresponding to the match. All the predicates in the rule (R-Preds) must be satisfied for the rule to return true. For the query above, the Q-Pred `updatesField` would be matched with the corresponding rule to evaluate the rule body.

Fact comparison: The R-Preds are matched against the facts stored in the knowledge base. For a rule to be satisfied, each R-Predicate in the body must find a corresponding fact in the knowledge base. In the above example, the fact that the NF `firewall_kern/xdp` updates the `ipv4.dst` field in the packet header would satisfy the rule for `updatesField`, abstracting the specific node `node_3`. The underscore ('_') in the rule serves as an anonymous variable, as the value of the node ID is not relevant for the match. This abstraction simplifies the query process by focusing only on whether the field *Fld* is updated by the *Nf*, regardless of the specific block where it is updated.

Handling variables: If the query contains variables, the engine retrieves all values that satisfy the rule from the knowledge base (i.e., replacing the retrieved values would satisfy the R-Predicate of the rule's body). For instance, if the above mentioned query is modified to ask which NFs update `ipv4.dst` (*i.e.*, `updatesField(Nf,"ipv4.dst")`), the engine will return `firewall_kern/xdp` as it satisfies the fact that it NF updates the destination IP field.

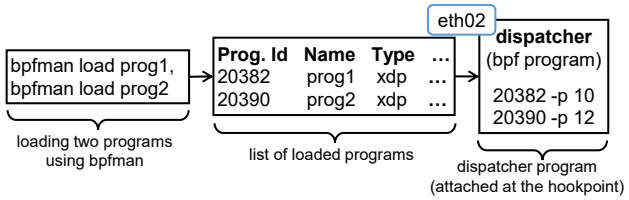
Result compilation and chaining: After the matching process, results are compiled by collecting all the values that satisfy the query predicates. If a query involves multiple predicates, each of them is evaluated sequentially by following a logical AND/OR operation. For example, if a query asks for NFs that both update a field (`updatesField`) and access a specific protocol (`accessesProtocol`), the engine first identifies NFs that update the field and then filters those based on protocol access, ensuring that only NFs meeting all criteria are returned.

A.7 Retrieval and Assertion Queries

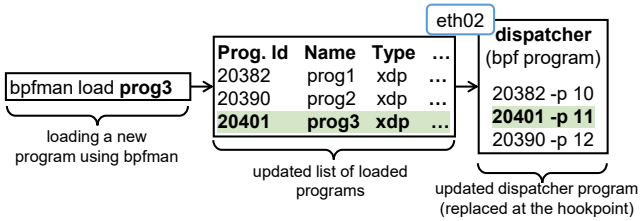
Table 6. List of Retrieval and Assertion Queries. The asterisk (*) denotes that the queries contain recursive predicates (*i.e.*, successorNF and predecessorNF).

Retrieval Queries		Assertion Queries	
Q1(R)	mapWrite(**Nf**, _ , **Fld**).	Q1(A)	mapWrite(<**NF3**>, _ , **ipv4.src**).
Q2(R)	updatesField(**Nf**, **Fld**).	Q2(A)	updatesField(<**NF8**>, **ipv4.dst**).
Q3(R)*	predecessorNF(**Nf**, **SNf**), passes(**SNf**, **Hook**, **[Fld, Arg]**).	Q3(A)*	predecessorNF(<**NF15**>,<**NF10**>), passes(<**NF10**>, **xdp**, [*,*]).
Q4(R)*	updatesField(**Nf**, **xdp_md.data**), successorNF(**Nf**, **SNf**), readsField(**SNf**,**xdp_md.data**).	Q4(A)*	updatesField(<**NF15**>, **ipv4.src**), successorNF(<**NF15**>, <**NF12**>), readsField(<**NF12**>, **ipv4.src**).
Q5(R)*	updatesField(**Nf**,**Fld**), successorNF(**Nf**,**SNf**), readsField(**SNf**,**Fld**).	Q5(A)*	updatesField(<**NF10**>, **xdp_md.data**), successorNF(<**NF10**>, <**NF12**>), readsField(<**NF12**>,**xdp_md.data**).
Q6(R)	readsField(**Nf**,**Fld**), readsField(**Nf**,**Fld**).	Q6(A)	readsField(<**NF6**>,**tcp.flags.syn**), readsField(<**NF6**>,**tcp.flags.ack**).
Q7(R)	callsHelper(**Nf**, **Helper**).	Q7(A)	callsHelper(<**NF3**>, **bpf_map_update_elem**).
Q8(R)	accessesProtocol(**Nf**,**Fld**,**Proto**), drops(**Nf**,**Hook**,**[Fld,Arg]**).	Q8(A)	accessesProtocol(<**NF15**>,**eth.type**,**ipv4**), drops(<**NF15**>, **xdp**,[*,*]).

A.8 Bytecode deployment using bpfman



(a) Two eBPF programs are attached with priorities (-p) 10 and 12.



(b) A new eBPF program is attached with priority (-p) 11.

Figure 9. Workflow of bpfman tool for attaching multiple eBPF programs to a hook point (i.e., "eth02" interface).

Bpfman is a tool to manage eBPF programs on local hosts and Kubernetes. It supports load, unload, attach, and detach eBPF programs. It also supports the coexistence of multiple eBPF programs on a single interface with the help of the libxdp library [8]. The workflow of bpfman is illustrated in the Figure 9. The eBPF bytecodes are loaded into the kernel and attached to a specific hook point. At a hook point, bpfman uses a special management program called a dispatcher to execute the programs in a specific order based on the priorities (a small number has high priority) set by the operator for each bytecode. The dispatcher program maintains a configuration structure to record the number of attached bytecodes, their priorities, chain call actions, and other metadata.

The chain call action decides whether the packet should continue processing or the processing should be terminated. When a new bytecode is attached, a new dispatcher is created with the updated list of programs. The system atomically swaps the old dispatcher with the updated one, ensuring seamless updates and enforcing the correct execution order without downtime.

A.9 CFG generation time and memory overheads

A.10 Comparison Overview

We compared Yaksha-Prashna with two state-of-the-art works: Klint [70] and DRACO [61]. Klint is a verification tool that checks the conformance of NF binaries with the specification written in Python. NF can be written in different languages, such as C or Rust, using frameworks such as DPDK or eBPF. DRACO is another verification tool that verifies eBPF-based NFs using specifications and assertions, but requires the

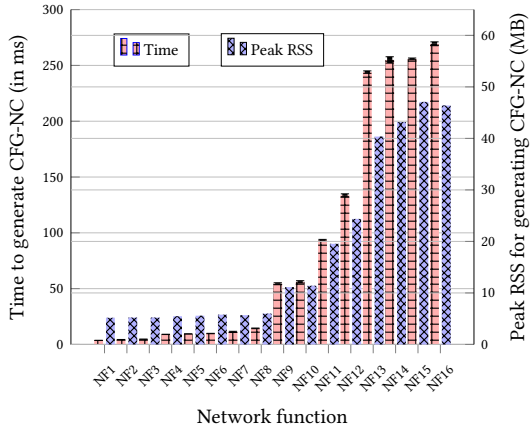


Figure 10. Time (left y-axis) and Memory (right y-axis) consumed for generating CFG-NC

availability of source code to do so. The comparison is organized into two parts: one dedicated to Klint and the other to DRACO, each addressing the corresponding class of NFs and properties used for verification.

A.10.1 Details of implemented NFs and properties extracted for comparison with Klint. For comparison, we evaluated NF's conformance to their specifications, similar to Klint's verification of C-based binaries. For this comparison, Yaksha-Prashna needed (1) eBPF bytecode of the NFs and (2) the properties used for the verification. Since Klint's NFs are written in C, we wrote semantically equivalent eBPF-based NFs and used their bytecode for comparison. These are six representative NFs: Router, NAT, Firewall, Bridge, Load Balancer, and Policer. We validated semantic equivalence between the original and rewritten NFs; the validation methodology is described in Appendix A.13. The properties to verify were extracted from the Klint specification. All properties fall into Category 1 (§2.2). Detailed description of the developed NFs and the extracted properties is discussed below. The Yaksha-Prashna assertions used to assert these properties can be found in Table 7.

(1) Router: The developed router is a basic IPv4 router that forwards the valid IP packets based on the routing table. The routing table is implemented using an eBPF map that maps the destination IP addresses to the interface. The router was verified using the following key properties:-

Property 1: Expected header order. An IPv4 router should only allow Ethernet followed by IPv4 packets. All other packets (e.g. IPv6 packets) must be dropped. This property helps to take protocol-specific packet actions. To verify this property, we asserted a high-level assertion, *"Only Ethernet and IPv4 packets should be forwarded"*, whose exact assertion is presented in row P1 of Table 7.

Property 2: Validity of IPv4 header. A router must ensure the validity of an IPv4 header before processing the packet. This is achieved by verifying properties such as: (1) the IP header length is at least 20 bytes, (2) the IP header length does

Table 7. Assertions for the properties and Retrieval Queries written using Yaksha-Prashna Language.

S.No.	Properties (P1-P22)/Retrieval Qs (Q23-24)	Yaksha-Prashna assertions/Retrieval Qs
P1	Expected header order	`!passes("nf_id", xdp, [(var, var)]), accessesProtocol("nf_id",_, "eth"), accessesProtocol("nf_id","eth.type","ipv4").`
P2	Validity of IPv4 header	`!passes("nf_id", xdp, [("ipv4.ihl", >= 5),("ipv4.ihl", <= "ipv4.totlen"), (...)]).`
P3	Expected header field updates by NF	`!passes("nf_id", xdp, [(var, var)]), "ipv4.ttl").`
P4	Longest prefix matching	`!passes("nf_id", xdp, [(var, var)]), mapLookup("nf_id", "router_map").`
P5	Packet content preservation	`!updatesField("nf_id", *).`
P6	Filtering external flows	`!passes("nf_id",xdp,[("xdp_md.ingress_ifindex", EXTERNAL), (var, var)]), mapLookup("nf_id","firewall_map").`
P7	Remembering internal flows	`!passes("nf_id", xdp, [("xdp_md.ingress_ifindex", !EXTERNAL),(var, var)]), mapWrite("nf_id", "firewall_map", *).`
P8	Update the source IP and port of the internal flow	`!passes("nf_id", xdp, [("xdp_md.ingress_ifindex", !EXTERNAL), (var, var)]), updatesField("nf_id",ipv4.src), updatesField("nf_id", "tcp.sport"); updatesField("nf_id", "udp.sport").`
P9	Restore the destination IP and port of the external flow	`!passes("nf_id", xdp, [("xdp_md.ingress_ifindex", EXTERNAL),(var, var)]), updatesField("nf_id", "ipv4.dst"), updatesField("nf_id", "tcp.dport"); updatesField("nf_id", "udp.dport").`
P10	Selected transmission port must not be the ingress port	`!passes("nf_id", xdp, [(var,var)]), mapLookup("nf_id", "bridge_map"), readsField("nf_id", "xdp_md.ingress_ifindex").`
P11	Broadcast if unknown	`!drops("nf_id", xdp, [(var, var)]), mapLookup("nf_id", "bridge_map").`
P12	The learning process	`!passes("nf_id", xdp, [(var, var)]), mapWrite("nf_id", “bridge_map”, *).`
P13	External traffic policing	`!drops("nf_id", xdp, [("xdp_md.ingress_ifindex", !EXTERNAL)]).`
P14	Backend liveness	`!drops("nf_id", xdp,[("xdp_md.ingress_ifindex", !EXTERNAL), (var, var)]), callsHelper("nf_id", "bpf_ktime_get_ns"), mapWrite("nf_id","backend_map",*).`
P15	Load balancing external traffic	`!passes("nf_id", xdp, [("xdp_md.ingress_ifindex", EXTERNAL), (var, var)]), mapLookup("nf_id", "backends_map").`
P16	Drop all fragmented packets	`passes("nf_id", xdp, [(var, var)]), readsField("nf_id", "ipv4.frag").`
P17	Forward all ICMP echo request packets	`passes("nf_id", xdp, [(var, var)]), readsField("nf_id", "icmp.request"), updateField("nf_id", "icmp.*"), updatesField("nf_id", "ipv4.*"), updatesField("nf_id", "eth.*").`
P18	Handle only TCP SYN packets	`passes("nf_id", xdp, [(var, var)]), readsField("nf_id", "tcp.syn").`
P19	Add custom redirection header	`passes("nf_id", xdp, [(var,var)]),callsHelper("nf_id","bpf_xdp_adjust_head"), updatesField("nf_id","tcp.dataoffset").`
P20	Forward all packets unchanged	`passes("nf_id", xdp, [(*, *)]). AND !updatesField("nf_id", *).`
P21	Correlated maps	`correlatedmaps(["map_one", "map_two"]).`
P22	NF dependency: Read after write (RAW), Write after read (WAR) and Write after write (WAW)	RAW assertion: `updatesField("nf_id1", *), successorNF("nf_id1", "nf_id2"), readsField("nf_id2", *).` WAR assertion: `readsField("nf_id1", *), successorNF("nf_id1", "nf_id2"), updatesField("nf_id2", *).` WAW assertion: `updatesField("nf_id1", *), successorNF("nf_id1", "nf_id2"), updatesField("nf_id2", *).`
Q23	Find the list of packet fields updated by NF	`updatesField("nf_id", Fld).`
Q24	Find the list of packet fields where one NF updates and its successor reads (RAW dependent)	`updatesField(“nf_id1”, Fld), successorNF(“nf_id1”, “nf_id2”), readsField(“nf_id2”, Fld).`

not exceed the total length of the packet, (3) the checksum is correct, and (4) the TTL is non-zero, etc. These validity checks are crucial for identifying and discarding malformed IP packets. We used high-level assertions such as *"Every outgoing packet must have an IP header length of at least 20 bytes"*, *"Every outgoing packet must have an IP header length less than total length"*, etc, to verify these properties. The exact assertion is presented in row P2 of Table 7.

Property 3: Expected header field updates by NF. One of the key properties of a router is to decrement the time-to-live (TTL) field of every outgoing packet. This property ensures that the packet does not circulate indefinitely within the network. We verified this property using the high-level assertion, *“IPv4 ttl must be updated on every outgoing packet”*, whose exact assertion is presented is row P3 of Table 7.

Property 4: Longest-prefix-match. An IPv4 router must perform longest-prefix-match (LPM) while mapping the destination IP address to the appropriate forwarding interface. LPM enables faster and more efficient matching than alternative approaches such as exact matching. eBPF provides a specific map type to perform LPM matching. To verify this property, we wrote two high-level assertion, *“All the outgoing packets must perform a lookup on router map"*, and *"The map type of the router map must be LPM”*. Since the Yaksha-Prashna language lacks the predicate to assert for the map

types, we could only use the first assertion for verification. The exact assertion is presented in row P4 of Table 7.

(2) Firewall: The developed firewall implements stateful traffic filtering, where the state is captured using a map. It primarily performs two tasks: (1) blocking unknown external flows, and (2) remembering internal flows, when necessary, and forwarding them. To verify the correctness of the firewall, we used the following three key properties:-

Property 5: Packet content preservation. Typically, a firewall should not modify the contents of the packets it forwards. This property is important for ensuring the integrity of the forwarded packets. We used the assertion A1, discussed in the motivation section (§2.2), to verify this property, which is also presented in row P5 of Table 7.

Property 6: Filtering external flows. One of the key properties of a firewall is to block unknown external packets. This property is crucial as it ensures the safety of the internal network. To verify the same, we asserted, *"All external packets must not be forwarded, if no entry is found in the firewall map"*. The exact assertion is presented in row P6 of Table 7.

Property 7: Remembering internal flows. For a stateful firewall to function properly, it must remember the state of the internal flows. This property is also crucial for allowing the "remembered" external flows (Property 6) in the network. To assert this property, we used a high-level assertion, *"For all internal packets, if the flow entry is not present in the firewall map, the map must be updated with flow information"*. The exact assertion is presented in row P7 of Table 7.

(3) NAT: The developed stateful NAT performs two primary functions: (1) for internal flows, it updates the source IP address and port with the public IP address and the masked port number, and (2) for external flows, it restores the destination IP address and port to the original internal IP address and port number. We also verified **Property 6** and **Property 7** on NAT, since they are essential for ensuring the safety of the internal network and guaranteeing the correct stateful behavior of the NAT. Apart from that, two more key properties were verified:-

Property 8: Update the source IP and port of the internal flow. For any internal flow, the NAT must update the source IP address to the public IP and mask the source port. This property is important because it allows multiple internal devices to be represented by a single public IP address, while still keeping their flows uniquely identifiable with the help of the masked port number. To verify the same, we used a high-level assertion, *"Source IP address and port of all the internal packets must be updated"*, whose exact assertion is presented in row P8 of Table 7.

Property 9: Restore the destination IP and port of the external flow. For any external flow, the NAT must restore the destination IP and port to the original internal private address and port. This is important to ensure that the external packets reaching NAT can be directed to the correct internal device. To verify the same, we used a high-level assertion, *"Destination IP address and port of all the known external packets must be updated"*. The exact assertion is presented in row P9 of Table 7.

(4) Bridge: The developed bridge is a MAC-learning bridge that forwards the Ethernet frame based on the destination MAC address. The mapping of the destination MAC and interface is stored using a map. The bridge was verified using the following three key properties:-

Property 10: Selected transmission port must not be the ingress port. For a bridge, the selected transmission port must not be the same as the ingress port of the received frame. This check is essential to prevent unnecessary loops in the network. We verified this property using the high-level assertion *"The transmission port of the outgoing packet must not be equal to the ingress port"*. The exact assertion is presented in row P10 of Table 7.

Property 11: Broadcast if unknown frame. If the frame received by the bridge does not match any entry in the forwarding table, then the frame must not be dropped and should be broadcast to all the interfaces, except the ingress interface. This property guarantees that the frame can still reach its intended destination, even when the bridge lacks knowledge of the appropriate outgoing interface. To verify this, we used two high-level assertions: *"If map lookup fails on the bridge map, then the packet must not be dropped"*, and *"The packet must be broadcast, except for the ingress"*. Since the Yaksha-Prashna Language does not support predicates to assert for packet broadcast, we could only verify the first assertion. The exact assertion is presented in row P11 of Table 7.

Property 12: The learning process. The learning process observes the source addresses of frames received on each port and updates the forwarding table. This process is important as it allows the bridge to build the forwarding table dynamically and forward frames efficiently, rather than relying on broadcasting. We verified this property using the high-level assertion, *"If the source MAC of the received frame is not present in the bridge map, then the map must be updated accordingly"*. The exact assertion is presented in row P12 of Table 7.

(5) Policer: The developed policer is a stateful network function that implements a per-destination token bucket for external packets, effectively limiting traffic based on the packet size. A map is used to keep track of the state, using the IP address as the key and storing the token bucket details, such as token size, as the associated value. We verified the following property on the policer:-

Property 13: External traffic policing. The policer should apply rate limiting only to external traffic, updating the state according to the packet size, while all internal packets must be forwarded. To verify the former property, we used the high-level assertion *"All external packets must perform lookup*

on the policer map and update it accordingly", while the latter property was verified using *"All internal packets must be forwarded"* assertion. The exact assertion is presented in row P13 of Table 7.

(6) Load balancer: The developed load balancer is an eBPF implementation of Google's load balancer called "Maglev". It picks one of the backends from the backend pool based on consistent hashing, and remembers the flow and backend mapping using a map. It also uses another map to keep track of the live backends, which is updated after receiving *heartbeat* packets from the backends. We verified the load balancer with **Property 5**, as the load balancer should not modify the packet contents while processing the packet. Apart from that, we also verified the following two key properties:-

Property 14: Backend liveness. If the packet comes from backends, it must update the live backend map with the current time and then get dropped. This is important to keep track of live backends and to avoid forwarding packets to dead backends. We verified the property using the high-level assertion, *"All the dropped backend packets must update the live backend map with the current time"*. The exact assertion is presented in row P14 of Table 7.

Property 15: Load balancing external traffic. The external packets should be forwarded to one of the chosen backends, and if the backend is dead, the packet must be dropped. We verified the property using the high-level assertion, *"All external packets must be forwarded to the backend if the backend is alive"*, whose exact assertion is presented in row 15 of Table 7.

A.10.2 Details of NFs and properties extracted for comparison with DRACO. We compared Yaksha-Prashna with DRACO in terms of expressiveness. We extracted the properties used by DRACO for verifying the three real-world eBPF programs, Katran [64], CRAB [53], Fluvia [2], and one eBPF program from academic literature, hXDP FW [22]. All the properties fall under Category 1 (§2.2). A brief description of the NFs and the extracted properties is discussed below. The Yaksha-Prashna assertion can be found in Table 7.

Katran: Katran is a Layer-4 load balancer developed by Meta that uses XDP for efficient packet processing. It is designed to provide scalable and low-latency connection balancing. Katran was verified using the following two key properties:-

Property 16: Drop all fragmented packets. One of Katran's constraints is its inability to handle fragmented packets; as a result, it drops all such packets. We expressed this property using a high-level assertion, *"None of the fragmented packets should be forwarded"*, whose exact assertion is presented in row P16 of Table 7.

Property 17: Forward all ICMP echo request packets. Katran takes many protocol-specific packet actions. One such action handles ICMP echo request packets. When encountering such request packets, Katran converts them into echo replies by updating the relevant ICMP, IP and Ethernet header fields, and forwards them. We expressed it using *"All the ICMP echo request packets must be forwarded after updating relevant fields of ICMP, IP and Ethernet header"* high-level assertion. The exact assertion is presented in row P17 of Table 7.

CRAB: CRAB is another Layer-4 load balancer that participates only during connection setup. It handles SYN packets to choose a backend server and lets clients and servers talk directly via *Connection Redirection*, thus lowering latency and avoiding bottlenecks while allowing complex stateful load balancing. We verified CRAB using the following two key properties:-

Property 18: Handle only TCP SYN packets. CRAB participates only in the connection establishment phase, where it handles TCP SYN packets and discards all the other packets. To express this property, we used the high-level assertion, *"All forwarded packets must be TCP SYN packets"*, whose exact assertion is presented in row P18 of Table 7.

Property 19: Add custom redirection header. To stay off the datapath between the client and the server, CRAB leverages *Connection Redirection*, where the communication between client and server happens directly without involving the load balancer. A custom redirection header is appended to all the outgoing packets to enable this feature. We expressed the property using the high-level assertion, "All the outgoing packets must have a custom header appended". The exact assertion is presented in row P19 of Table 7.

Fluvia: Fluvia is an IPFIX exporter that leverages XDP to collect metadata related to the incoming IPv6 flows, and exports it to the userspace for analysis. Fluvia was verified using the following property:-

Property 20: Forward all the packets unchanged. A flow exporter should transparently collect the flow's metadata without dropping any incoming packets. Moreover, it should also not modify the packet contents. We used the high-level assertion, *"All packets must be passed"*, in conjunction with **Property 5** to express the same. The exact assertion is presented in row P20 of Table 7.

hXDP Firewall: hXDP firewall is a simple stateful firewall from the academic literature[22], which implements similar functionality as the firewall discussed in Section §A.10.1. Apart from verifying the key firewall properties (Property 5-7), we also verified the following specific property, which is specific to the hXDP firewall implementation:-

Property 21: Correlated Maps. Correlated maps are often used to share state across different maps. Two maps are considered correlated when the return value of one map operation is used as the argument for another map operation, such as a lookup or update. Lines 13-15 of Listing 1 capture an instance of correlated maps, where the value from map lookup on *flow_ctx_table* is stored in *flow_leaf* (line 13),

and later used as a key for redirecting the packet using the *tx_port* map (line 15). We have a dedicated map predicate *correlated_maps* to capture the correlated maps, which is presented in row P21 of Table 7.

In addition to the aforementioned properties, DRACO also asserted a few generic network function properties, such as "Ethernet source address remains unchanged across all packets", "All IPv6 packets should be dropped", etc. These properties could likewise be expressed using the Yaksha-Prashna Language.

A.11 Query execution

Number of Q-Predicates. The more predicates in the query, the more facts in the knowledge base have to be traversed to resolve the query and, hence, the more time it takes for the query to complete. For instance, among two non-recursive queries, Q1 and Q6, with one and two Q-Predicates, the former takes less time (0.59μs) than the latter (2.5μs) to complete when executed on the facts obtained from a chain of sixteen NFs. Similarly, Q3 with two Q-Predicates finishes faster (1.4μs) than Q4 and Q5, which have three Q-Predicates (1.6μs and 3.3μs). We also observed that the combination queries with more than three Q-Predicates (Q(3,4), Q(4,5), Q(3,5), Q(3,4,5) and Q(6,7,8)) took a longer time (6.6μs, 8.9μs, 4.7μs and 9.6μs) comparatively to complete than the queries with three or less Q-Predicates (Q(1,2) and Q(7,8)) which took comparatively less time (1.48μs and 2.25μs) when executed on the facts corresponding to the chain of sixteen NFs. Additionally, the time grows linearly with the #Q-Predicates.

NF-Chain length. We observe that the number of instructions to be analyzed increases as the NF chain length increases, resulting in a large set of facts being generated for each NF in the chain. Consequently, in general, the query execution time of both individual and combination queries increases[10]. In other words, the longer the NF chain, the more time it takes to execute the queries. Execution of all the 15 queries take 27.9μs, 33.2μs, and 48.8μs on the facts generated by 2, 8, and 16 NF-chains. We show the time taken by different queries and the predicates within each query on the facts generated by each of these NF-chains in Figure 6.

A.12 Memory footprint

We evaluate the memory footprint to understand the memory utilized by our system for generating CFG-NC and executing queries. We use peak Resident Set Size (RSS) in MBs as a metric for computing memory footprint. The peak RSS is the maximum memory consumed by the process during its lifetime. We use the GNU `time` command [3] to measure the peak RSS value. We compute memory footprint while (i) generating CFG-NC of the microbenchmark NFs listed in Table 4, and (ii) executing individual assertion queries (Q1–Q8) and their combinations listed in Table 6.

Peak RSS for CFG-NC generation. We show the peak RSS values obtained while generating the CFG-NC in Figure 10 (right). We analyzed the correlation of peak RSS with the number of paths in the Control Flow Graph (CFG) and the number of instructions analyzed (NAI). We observe that memory usage increases with the number of paths and instructions. Importantly, our system consumes less than 50 MB for the largest microbenchmark NFs (NF13–NF16).

Peak RSS for Query execution. We observe that the peak RSS size remains almost constant for answering different queries for a given set of facts. Specifically, for the facts generated from all the 16 NFs, the Query Engine consumes 13.25–13.42 MB of memory for answering our queries.

Comparison with Klint. Figure 7 (right) shows peak RSS memory usage during verification. Klint requires 190–1000 MB, while Yaksha-Prashna uses only 20-50 MB. Despite storing the network context of the entire bytecode, Yaksha-Prashna consumed 90-95% less memory than Klint.

A.13 Semantic equivalence of rewritten eBPF programs for comparison with Klint

To ensure semantic equivalence in our comparison with Klint, we followed a three-step methodology.

Logic-preserving rewriting. We manually rewrote Klint's C-based NFs as XDP-compatible eBPF programs. During this process, we carefully preserved the original packet-processing logic, control flow, and state updates. The rewritten programs were manually inspected to confirm that they implement equivalent packet-processing behavior.

Equivalent simplification. In cases where eBPF constraints prevented direct translation of certain components (e.g., complex hash functions), we applied equivalent simplifications to both implementations. For example, we replaced complex hash functions with simpler alternatives in both versions to maintain comparable computational complexity while preserving functional intent.

Configuration alignment. We aligned configuration parameters across implementations, including the number of interfaces and table/map sizes, to avoid discrepancies arising from differing resource allocations.

We note that program equivalence is *undecidable* in general. Our validation therefore relies on careful manual inspection and controlled alignment rather than formal equivalence checking.

[10]There can be some exceptions due to the optimizations on indexing and caching mechanism of the Prolog runtime engine.